

The Ledger: An Orchestrator of Smart Contracts

One Design, Shown on Real Products

R. Delloye

July 2026

Abstract

Every firm that trades keeps score more than once. One set of systems books the trades; another values them; a third reports them; a fourth settles them. These systems hold overlapping copies of the same facts, and the copies drift apart. When they disagree — and they disagree every day — people reconcile: they find the breaks, decide which copy is right, and correct the others by hand. This document describes a system of record for post-trade activity built so that a large class of those disagreements cannot occur, because the numbers they would disagree about are never stored twice. Positions, profit and loss, balance sheets, and reports are computed from one immutable record rather than maintained alongside it.

The design rests on a single claim. *The ledger is the orchestrator of the output of many smart contracts.* The smart contracts run outside the ledger’s trust boundary, each triggered by an event — an incoming trade, a lifecycle event (an expiry, a coupon, a barrier fixing), or a corporate action (a dividend, a split, a spin-off). When its trigger fires, a contract reads the declared data in the ledger and the stamped observations it is allowed to see, works out what must change, and produces transactions — moves, terms versions, status writes. The ledger runs none of the contracts’ product-specific logic. Transactions arrive at its single door; the door admits each one, records it in the immutable log, and every view — positions, profit and loss, reports — is computed from the log.

The chapters that follow substantiate this claim: first the door and the log, then the declared data a contract may read, then the contracts and their triggers, and finally a sequence of real products — cash equity, a dividend forecast, listed futures, a structured note, a special dividend, a spin-off, an elective merger, and a securities loan — each processed by the same machine.

Contents

1	The Orchestrator Thesis	4
1.1	The problem is reconciliation	4
1.2	The division of labour	4
1.3	Three words that need pinning down	4
1.4	Trusted for nothing, verifiable in everything	5
1.5	Two layers of correctness	6
2	The Door and the Log	6
2.1	Wallets, units, and the atomic move	7
2.2	Conservation, by construction	7
2.3	The single door and what it checks	8
2.4	The log, and why untrusted proposals are safe	9
2.5	Every view is a fold of the log	9
2.6	Time travel re-reads; it never re-executes	10

3	Declared Data	10
3.1	Terms, and versions that only ever append	10
3.2	Four dimensions, and how a field reacts	11
3.3	The adjustment schedule and its three layers	12
3.4	The datum-kind registry	13
3.5	Operators: one declaration, two consumers	14
4	The Smart Contracts and Their Triggers	15
4.1	One architecture, three triggers	15
4.2	What the contract layer is required to satisfy	16
4.3	Two layers of correctness, on numbers	17
4.4	Provenance is recorded, never acted on	18
4.5	The model-agnostic wall	19
5	The Product Tour	19
6	Cash Equity and the Split Phantom	20
6.1	The trade and its dividend	20
6.2	The split, and the phantom it invites	21
7	A Dividend Forecast as a Two-Component Datum	22
7.1	One datum, two components	22
7.2	One wrong word, half the money	23
8	Listed Futures Daily Settlement	23
8.1	The life of the contract	24
8.2	Margin is just conservation	24
9	A Structured Note, Model-Free	25
9.1	The coupon: a scheduled cash move	25
9.2	The barrier: an observation, not an opinion	25
9.3	Expiry, and the two layers on one number	26
10	A Bespoke Special-Dividend Override	26
10.1	Where the default and the term sheet disagree	27
10.2	Three layers, and a gate that will not improvise	27
11	The Spin-Off	28
11.1	One transaction across two lines	28
11.2	A mark carried only as far as the data supports	29
12	An Elective Merger	29
12.1	Nothing moves until confirmation	30
12.2	Different moves, one fate, and a weld in aggregate	30
13	Securities Lending in Brief	31
13.1	A loan, opened and recalled	31
13.2	The special dividend, split by payer	32
13.3	Recall, with a deadline	33

14 What the Full Framework Adds	34
14.1 Regulatory reporting is a projection, not a second record	34
14.2 Alignment to a common domain model	34
14.3 The correctness claims are executable, not asserted	35
14.4 The execution engine and the liveness it underwrites	35
14.5 What this framework does not do	36
15 Conclusion	36
15.1 The thesis, now shown	36
15.2 Each stop, and the choice it settled	38
15.3 What the whole design rests on	38
15.4 The formal synthesis	39

1 The Orchestrator Thesis

1.1 The problem is reconciliation

Consider what has to be true for two internal reports to agree. A trading desk's risk system says the book holds 2,000 shares of a company; the firm's general ledger says the same position is worth €100,000; the regulatory report says a dividend was received on it last week. These three statements are about one underlying fact — a holding and its history — but each system stores its own copy of that fact and updates it on its own schedule. If the risk system applied a stock split and the general ledger did not, the share count and the valuation now describe different states of the world. Nothing inside either system is wrong; they are simply out of step. Finding that break, and fixing it, is reconciliation, and it is where much of the cost and most of the operational risk of a post-trade system lives.

The design in this document removes the source of the break rather than detecting it faster. There is one record: an ordered, append-only stream of events. A position is not a stored number that systems update; it is the sum of the moves that touched it, computed on demand from the stream. Profit and loss is not a second stored number; it is the difference of two valuations, each computed from the same positions. A balance sheet is a projection; a report is a projection. Because balances, profit and loss, and reports are all computed from the one stream, two of them cannot disagree about a fact the stream records only once. Within the system's scope, three reconciliation failures that dominate real operations — trading system against general ledger, profit and loss against balance sheet, and one entity's books against another's — are not merely made rare. Each of them is a disagreement between two projections of one record, and a disagreement between two projections of one record is not a state the system can be in.

1.2 The division of labour

The stream is not enough on its own, because someone has to decide what events to put in it. A trade must become a change in holdings and cash; a coupon date must become a payment; a two-for-one split must double a share count and halve a price reference. Each of these decisions is specific to a product and, often, to a single contract's terms. There is no general rule that turns “a barrier was observed at 78.40” into the right set of moves for every structured note ever written; the note's own terms say what happens.

The design draws a sharp line here. The logic that works out what a particular event means for a particular product lives in a *smart contract*: a deterministic function that reads the relevant facts and produces the resulting transactions. A financial instrument, on this view, is a contract that emits moves, and adding a new product is writing a new contract, not changing the ledger. The ledger itself performs none of this product-specific work. It admits transactions; it does not manufacture them. Its job is to take each proposed transaction at its single door, decide whether the transaction is well-formed, record it if so, and refuse it otherwise — and then to answer every question about the resulting state by reading the log.

This division is what the word *orchestrator* in the thesis names. The contracts compute; the ledger admits and records; the views are read back. The ledger orchestrates the output of the contracts without running the contracts' internals, in the same sense that a database commits the rows an application hands it without containing the application's business rules.

1.3 Three words that need pinning down

The thesis uses three phrases loosely enough to mislead a careful reader, so we fix their exact content before building on them.

“Outside the ledger” means outside its trust boundary. A contract is external to the ledger, untrusted by it, and verifiable against it. None of this requires the contract to run in a separate process or on a separate machine. The reference implementation of a processor is an ordinary function called in the same process as the door. “Outside” is a statement about authority, not location: the ledger never treats a contract’s output as authoritative merely because the contract produced it. Whatever a contract submits is a proposal until the door admits it, and it is admitted under exactly the same checks as every other transaction.

“Runs none of the logic” is scoped to product-specific logic. The ledger interprets nothing and prices nothing. It runs no pricing model, no product-specific settlement rule, no event-specific special case. It does, however, run one generic thing, and honesty requires naming it. When a datum recorded before a corporate action is read after it, the ledger converts that datum by an operator the corporate action *declared* — halving a price across a two-for-one split, for instance. This conversion is not product logic and not a pricing decision; it is the mechanical application of a declared operator drawn from a small fixed menu, and the ledger applies it uniformly to everything that crosses the boundary. The pricing function that decided a note was worth what it was worth stays a black box the ledger never opens. Section 3 makes this precise; here it is enough to say that “the ledger runs none of the logic” means none of the *contracts’* logic. The one computation the ledger does perform is this read-seam application, and it is not an interpreter of anything a contract wrote.

Conservation is not something the door checks. It would be natural to say the door admits a transaction only after checking that it does not create or destroy value. That is not what happens, and the difference matters. The unit of change is a *move*: a positive quantity of one unit leaves one wallet and the same quantity enters another. A move is a transfer and nothing else; it has no way to express a quantity appearing from nowhere. Because a transaction is built from moves, an unconserved transaction cannot be constructed — there is no value for the door to reject. The door checks the things that *can* fail. Conservation is not among them, because it cannot fail. Section 2 develops this; it is the first and strongest instance of a principle the whole design leans on: where an illegal state can be made unrepresentable, that is preferred to checking for it, because a check can be forgotten and a type cannot.

1.4 Trusted for nothing, verifiable in everything

If the contracts are untrusted, why is it safe to record what they produce? Because everything a contract reads is in the log. A contract reads declared terms, the adjustment schedule, dimensions, the datum-kind registry, and stamped observations — all of them log-resident. Given the log up to the point where a contract ran, anyone can run the same computation again and check that the contract’s output is what the declared inputs imply. The ledger therefore trusts a contract to *produce* a transaction, never to be *right* about it; being right is not trusted, it is checked. This is what “trusted for nothing, verifiable in everything” asserts, and for everything a contract produces the assertion is unconditional.

For what the system takes in from the outside world, the assertion is not unconditional, and it would be dishonest to claim otherwise. Two inputs rest on named, bounded trust assumptions. First, when a market observation is stamped on the way in, the stamp is trusted to describe the source’s dissemination truthfully. Second, before a kind of observation may be consumed at all, it is registered, and the registration declares how that kind transforms across a boundary. That declaration is trusted — but only within a bound. A mandatory per-entry witness checks the declaration against the value-invariance identity and refuses any kind whose declared dimension would not survive a scale; the trust reaches exactly as far as that check and no further. These two assumptions are stated once, carried explicitly, and made to fail closed. They are not

eliminated. A further mechanism illustrates the limit, and it is a rule of the ledger, not a judgement about the market. Every stamped datum declares the basis it was observed in. If a datum asserts a basis for which no boundary has been recorded — it claims to live one frame forward of anything the log knows — the ingest door refuses it and quarantines the series by policy. The rule fires on a structural mismatch between the datum’s declared basis and the log, never on an economic guess that some price move “must” have been a corporate action. What lies outside the system is only the human follow-up: obtaining the missing attested notice, so the boundary can be recorded and the series released. That is detection and repair after the fact, not prevention, and the system is honest about the difference.

1.5 Two layers of correctness

The distinction between what is guaranteed and what is checked runs through the whole design, and it is worth stating once, plainly, as two layers.

Principle 1.1 (Admission and recomputation). *Admission guarantees that no transaction can break the ledger; recomputability from declared data guarantees that a wrong transaction can always be detected. Structural correctness is enforced at the door. Economic correctness is not enforced at the door: it is made checkable by recomputing the transaction from the log, and a failed check marks a producer defect, repaired downstream.*

The first layer is unconditional and owes nothing to the contract. A transaction that reaches the log conserves every unit, applies in full or not at all, writes each fact through its one legal writer, and leaves no dangling reference — and it has these properties whether the contract that produced it was careful, careless, or hostile, because the properties never depended on the contract. The second layer is where a contract can still be wrong: it can propose a transaction that is perfectly well-formed and yet books the wrong economics — pays €98 where the declared terms owe €100. The door has no basis to refuse such a transaction, and it does not try. Instead the discrepancy is caught by recomputing the transaction from the log and comparing; the difference is exactly the defect, and it is a defect in the producer, never in the ledger, repaired by a compensating transaction. The ledger does not manufacture that repair. The producer is corrected and resubmits through the same door, and the resubmission carries a `corrects` field naming the transaction it supersedes; the original stays in the log, now marked corrected, and both the error and its repair survive. Section 4 works this example through in full.

The two layers reduce to one sentence: the ledger *relies on* the first and *expects* the second. It relies on structural correctness because structural correctness is guaranteed by construction and no producer can break it. It expects economic correctness of its contracts, and rather than trusting that expectation it makes it checkable, turning “the contract must be right” from an article of faith into a recomputation anyone can run.

The remainder of the document is this claim, developed and then substantiated. Section 2 builds the door and the log and shows why an untrusted proposal is safe to record. Section 3 defines the declared data a contract may read. Section 4 specifies the contracts and proves out the two layers. The product tour that follows runs eight economically distinct events through the one machine, and each stop closes one design question by showing the mechanism absorb a real product with real numbers.

2 The Door and the Log

The ledger is a door, some checks, a log, views computed from the log, and one generic operation besides: at the read seam it applies a declared operator to pre-boundary market data, carrying an old number into a new frame. This section builds the door, the checks, the log, and the views from four primitives — wallet, unit, move, and transaction; the read-seam operation is the subject of Section 3. Together these are the whole of what the ledger does.

2.1 Wallets, units, and the atomic move

A *unit* is a thing that can be held — a share, a currency, a futures contract, a bespoke note. A *wallet* is a named partition of position space, and its entire state is one signed number per unit. A positive number is a long position; a negative number is a short. There is no separate machinery for shorts and no separate register of ownership standing beside the balance: economic ownership *is* the wallet balance. If a wallet holds +2,000 of a unit, it owns 2,000 of that unit. For an outright holding, long or short, the scalar balance is the whole story: there is nothing else to keep in step. The one exception proves the rule. A securities loan carries coordinates the balance alone does not hold — the borrow’s recall risk, its fee accrual, the manufactured dividend owed on the lent line — and those are not smuggled into the balance; they are added as their own units and moves at the securities-lending stop (Section 13). An outright long or short has no such coordinates, and for it the balance says everything.

The only way a balance changes is a *move*.

Definition 2.1 (Atomic move). *A move transfers a positive quantity q of one unit from a source wallet to a destination wallet in one indivisible step: the source decreases by q , the destination increases by q . The magnitude is positive; the direction is carried by which wallet is the source and which the destination.*

Two design decisions are folded into this definition, and both pay off later. First, the quantity is an exact integer in minor units — never a floating-point number. A share count is an integer; a cash amount is an integer number of cents. Division does occur — a withholding on an odd amount, cash in lieu of a fractional entitlement, an odd per-share dividend, a three-for-two on an odd lot — but it never leaves an unbooked remainder. The rounding convention is itself declared data, and whatever a division cannot distribute evenly is booked explicitly, as a cash-in-lieu or rounding leg that is a move like any other. The conserved quantities stay exact not because nothing divides but because the remainder is never dropped: it is recorded. Conservation and replay are therefore matters of integer arithmetic, decided exactly at any scale. Second, a move carries a single positive magnitude and two wallets. It has no field in which a quantity could be created or destroyed. Just as a move has no endpoint beyond the two wallets it connects, the door admits no outcome beyond success and refusal. Its Haskell signature says exactly that:

```
applyTx :: Transaction -> Ledger -> Either LedgerError Ledger
```

A transaction either produces a whole new ledger or a typed `LedgerError`. There is no third outcome and, in particular, no partially applied state.

A *transaction* is the atomic event recorded in the log: an ordered list of moves together with the non-balance changes that ride with them — bookkeeping rows, status writes, an introduction or appending of terms. It applies in full or not at all. Atomicity is the property that pays off here: a corporate action that must double a share count, halve a strike, and set a basis marker does all three in one transaction or none of them, so no reader can ever observe a half-applied event.

2.2 Conservation, by construction

Here is the law the whole design is organised around: for every transaction and every unit, the total quantity across all wallets is unchanged. Nothing is created and nothing is destroyed; value only moves.

The important thing is *how* this law holds. It is not checked at the door. Look again at what a move does: it subtracts q from one wallet and adds q to another, so its net contribution to the total quantity of its unit is $q - q = 0$. A transaction is a list of moves, and the total change it makes is the sum of the per-move contributions, each of which is zero. The sum is zero because every term is zero, not because anything counted it and found it balanced. There

is no transaction that fails to conserve waiting to be rejected — such a transaction cannot be built, because the only tool for changing a balance is a move, and a move is a transfer.

This is worth dwelling on because it is the pattern the design repeats everywhere. One could imagine a `set_balance` operation that writes a wallet’s balance directly, guarded by a runtime check that total value still sums to zero. That design has a hole: the check can be bypassed, disabled, or wrong, and the day it is, the books are unconserved and nobody knows. The design here has no `set_balance`. Opening a position is not a special primitive; it is an ordinary move from a *virtual wallet* that stands for the outside counterparty. Every counterparty outside the system is represented by a virtual wallet inside it, so every move has two endpoints inside the ledger and the system is closed. A conservation failure is therefore not a rejected financial event; it is an impossible one — and if it ever appeared it would be an implementation bug, not a transaction the ledger admitted.

2.3 The single door and what it checks

Every change to the ledger goes through one function. Registration goes through it; a trade goes through it; a corporate action goes through it. There is no second way in, no privileged channel, and no writer of a balance other than a move applied by this door. This matters precisely because contracts are untrusted: a contract cannot hold write access, because write access does not exist as a thing to hold. A contract can only submit a proposal, and a proposal becomes fact only by passing through the same door as everything else.

Since conservation cannot fail, the door does not check it. The door checks the things that *can* fail:

- **Basis agreement.** A quantity and a price may be combined only when they are expressed in the same frame. When a transaction records or consumes an observation, the door checks that its basis marker agrees with the state it is being joined to. This is the check that a stale, pre-split price is not silently multiplied against a post-split share count; Section 3 explains the frame it enforces.
- **Idempotence.** Each transaction carries an identifier derived from its cause. If the same transaction is submitted twice — a processor retried after a crash, a timer that fired late and then fired again — the door commits it once. A coupon due on one date is paid once even if its contract runs three times.
- **Legal status writes.** Facts other than balances — a unit’s lifecycle state, its basis marker, its last settlement — are written only through a small closed set of status writes, each applied by a single writer as an absolute replacement. A contract cannot invent a new kind of status write or write a field that is not its to write.
- **The invariance weld.** A transaction that carries a unit across a basis boundary is admitted only if its booked entitlement moves realise the boundary’s declared operator: for the boundary’s (f, c) , each holder’s quantity leg realises $q \mapsto q \cdot f$ and each cash leg equals $q \cdot c$. This welds an adjustment’s moves to the operator the boundary declares, so a transaction cannot claim a boundary and then book moves that do not match it. Later sections apply the same weld in two forms derived from this one — across two boundaries sharing an effective time, and per holder or in aggregate when holders elect — but the check is this one, stated here.

Alongside these, the door refuses a transaction that references a unit never registered, that points its cause at an event not yet committed, or that would leave the adjustment schedule incomplete — each a well-formedness condition, each returning a typed error and no state. The common thread is that the door never guesses and never repairs. A rejected transaction leaves

no trace in the ledger’s state and no entry in the log: `applyTx` returns either the whole new ledger or a typed error, and a typed error is a returned value, not a commit. The refusal carries the rejected payload back to whoever submitted it; at the datum ingest door this returned refusal *is* the quarantine, holding the payload as a value without ever storing it inside the system of record. If a deployment keeps a durable journal of what it refused, that journal is an operational artifact on the submitting side, never a second log within the ledger. Defaults fail closed: an absent declaration is a refusal, not an assumed identity; an unregistered unit is refused with its payload returned, not admitted on optimistic terms. The door would rather stop than improvise, because improvising is exactly how a system of record acquires a fact nobody can account for.

2.4 The log, and why untrusted proposals are safe

What the door admits, it appends to a log that is never rewritten. The log is ordered and hash-chained: no earlier entry is edited in place, and any tampering with history is evident to any party holding a prior chain head — anchoring that head outside the system is a deployment matter. Corrections are not edits; a mistake is repaired by a later, compensating transaction that is itself recorded, so the record of the mistake and the record of its repair both survive.

We can now answer the question the thesis raises: if contracts are untrusted, why is it safe to record what they propose? Because admission and recomputation together bound what a bad proposal can do. A defective contract cannot break an invariant — there is no unconserved proposal for the door to admit, no partial state it can leave, no row it can delete. The worst it can do is book wrong economics inside a well-formed transaction, and because everything it read is in the log, that error is not hidden: anyone can recompute the transaction from the log and find it. This detection is after the fact. The recomputation runs over the committed log, and nothing gates a view or the settlement projection on its having run first; the guarantee is that a wrong economics is always findable from the log and detected when the witness runs — which the deployment must schedule — not verified before it is consumed. What that buys is precise. The structural invariants are never breached — not for an instant, because they hold by construction, not by a check a wrong contract might slip past. The error is confined to the *economics* of the one well-formed transaction that carried it, and its repair is deterministic, with full lineage, through the compensating path. So the blast radius of a wrong contract is bounded above by “a detectable, well-formed, wrong transaction, repaired by a recorded correction”, which is a bounded and repairable thing. It is never “a corrupted ledger”, which is not. The gates that do act before consumption — the confirmation gate that will not let a datum cross an unarmed boundary (a boundary is armed only once its operator is confirmed), the pending quarantine on a datum with no operator to carry it across — guard boundary-crossing consumption, not a producer’s economics; they are not recomputation and must not be mistaken for it.

Consider the first transaction any book records: a purchase of 1,000 shares of a company at €100. It enters as one transaction with one move — 1,000 of the share unit from the counterparty’s virtual wallet into the book’s wallet — and a second move of €100,000 of cash the other way. The share count in the book’s wallet is now 1,000; the counterparty’s virtual wallet is –1,000, which is the reconciliation point against the counterparty’s own record of the same trade. Nothing here is a stored “position” that some later process must keep current. The position is 1,000 because the log says one move of 1,000 happened, and it will say so forever.

2.5 Every view is a fold of the log

A balance is the sum of the moves that touched a wallet. A valuation over a chosen set of wallets is the sum of their balances times an externally supplied price vector; profit and loss is the difference of two valuations. That difference depends only on the endpoints, not on the route

between them — but it measures performance only when no external contribution crossed into the wallet set between the two dates. A deposit or a withdrawal moves the valuation without being profit; performance is the endpoint difference *less* the net external flow over the interval. Inside the closed system every such flow is itself a recorded move across the wallet set’s boundary, so the adjustment is exact and computed from the log, never estimated. With flows accounted for, the endpoints alone fix the number. Value, unlike quantity, obeys no conservation law: a book can be worth more today than yesterday, and nothing is violated. What matters is that every one of these numbers is *computed* from the log when asked, not stored beside it. The unit-status cache the ledger keeps for speed is exactly that — a cache, derivable by replaying the events and safe to discard, never an authority. Replaying a unit’s events to any past point rebuilds the exact value that held then.

This is what makes the reconciliation failures of Section 1 unrepresentable rather than rare. The balance sheet and the profit-and-loss statement are two folds of one log. For them to disagree, one fold would have to see an event the other does not, and there is only one log.

2.6 Time travel re-reads; it never re-executes

Because the log is complete and the views are folds of it, the ledger can reconstruct its exact state as of any past moment: replay the stream up to that moment and stop. Balances, unit state, and the basis coordinate all come back exactly as they stood. This reconstruction re-reads recorded transactions; it never re-executes a contract, and it fetches nothing from outside the log. The consequence is worth stating precisely.

This is the property that makes an upgradeable, untrusted contract layer safe to live with: *a contract can be upgraded, replaced, or discarded, and every past state of the ledger still reconstructs exactly*, because reconstruction reads the transactions a contract once produced and never re-runs the contract that produced them. A contract’s output, once admitted, is a fact in the log like any other. Whether the contract still exists, still behaves the same way, or was thrown away years ago has no bearing on what the log says happened. Replay is deterministic and independent of where it is cut, so the same history reconstructs to the same state every time. The record does not depend on the machinery that fed it — which is the property a system of record must have, and the reason the ledger can hold untrusted output without being at the mercy of it.

3 Declared Data

A contract reads two things and nothing else: declared data and stamped observations. This section is about the first. Declared data is the part of the log that is authored deliberately, ahead of the events that consume it — the terms of an instrument, the way each of its fields reacts to a corporate action, the kinds of observation the system will accept, and the operators that carry an old number across a boundary into a new frame. The discipline throughout is the same one the door already showed: fix the answer in advance, as data, so that no event has to improvise.

3.1 Terms, and versions that only ever append

Every unit has *ProductTerms*: the terms of the instrument, recorded as a non-empty, append-only list of versions. The first version is written when the unit is registered; a unit that is registered but has no terms is not a state the types allow. There is no operation that edits a terms version, and — this is the part that matters for corporate actions — an adjustment never edits terms either. When a split halves a strike, it does not overwrite the strike field; it *appends* a new terms version carrying the halved strike, tagged with the boundary that caused it. The

old version stays exactly as it was written. Terms are declared data, and declared data, like everything in the log, grows only by appending.

3.2 Four dimensions, and how a field reacts

Why does a split halve a strike but not a coupon, and halve a cash-per-share dividend but not a percentage one? Because a field's reaction to a corporate action is fixed entirely by *what kind of quantity the field is*, and there are only four kinds. Every terms field declares, at the point it is defined, exactly one *dimension*:

- a **quantity** of the unit — a share count, a number of contracts;
- a **price** in the unit's basis — a strike, a stored reference spot;
- a **cash** amount — a fixed coupon, a notional;
- a **dimensionless** value — a ratio, a percentage, a participation.

The dimension is the field's constructor: a field with no dimension is not invalid, it is unconstructible. A corporate action declares a quantity factor $f \neq 0$ and a per-share cash amount c , and each dimension has one default action, the same for every product:

quantity $q \mapsto q \cdot f$, price $p \mapsto (p - c)/f$, cash and dimensionless: unchanged.

The factor is nonzero, so the boundary is invertible: a datum carried across it can be carried back. That is the whole of the constraint, and it is why $f = 0$ is refused — a zero factor is non-invertible, collapsing every quantity to zero with no way to recover the count it came from, so a datum read across such a boundary could never be transported in either direction. A negative factor is well-formed; only zero is refused.

Check the arithmetic against intuition. A two-for-one split is $f = 2$, $c = 0$: a holding of 1,000 becomes 2,000, a strike of €100 becomes €50, a fixed coupon in cash is untouched. The price action earns its keep on market data at an ordinary dividend. An ordinary €2 dividend is $f = 1$, $c = 2$; a market price printed at €102 the instant before the stock goes ex, read across the boundary, becomes $(102 - 2)/1 = €100$ — exactly the print going ex-dividend. That is the map doing its one job on a datum in the price dimension.

A price that is a *terms field* — a stored strike or reference spot on the instrument itself — is a different object, and under an ordinary dividend it does not move: it stands at €102. The arithmetic is the same; what differs is that applying it to a terms field here would pay the same cash twice, and the next subsection makes that precise. The dimensional action is where a field starts; the schedule, developed there, is what holds a price field fixed exactly where moving it would double-count.

The reason one rule suffices for every product is a small theorem about these four dimensions, and it is the load-bearing fact of the whole adjustment machinery.

Theorem 3.1 (Dimension invariance). *For a quantity q and a price p crossing a boundary (f, c) ,*

$$(q \cdot f) \cdot \frac{p - c}{f} = q \cdot p - q \cdot c.$$

The marked value after the boundary equals the value before, less exactly the cash the same boundary paid out.

The identity holds field by field, for every product, at every boundary on the menu, because it is quantified over the four dimensions once and not over products or event types. A product registered next year adds fields with dimensions; it adds no cases to this theorem. That is the sense in which the declarations *relocate* the expert judgement rather than removing it: someone must still say, correctly, which dimension each field of each product has — but having said it once, they never write another adjustment rule, and N products crossing M kinds of corporate action need N sets of field declarations, not $N \times M$ hand-written cases.

3.3 The adjustment schedule and its three layers

Declaring dimensions gives every field a *default* reaction. But real term sheets deviate. A specific note may say its strike adjusts on a special dividend only above a threshold; a specific determination may be left to a calculation agent to fix later. So every unit's terms carry, from the first version, an *adjustment schedule*: for every class of corporate action and every terms field, exactly one entry saying how that field reacts. An entry comes from exactly one of three layers, most specific winning:

1. the **class default** — fixed once, in this specification, per class and dimension: quantities scale by f , cash and dimensionless fields stand, and a price terms field moves to $(p - c)/f$ under every class *except* an ordinary dividend, where it stands. That one exception is neither market custom imported nor a consequence of Theorem 3.1 alone. The re-expression $p \mapsto (p - c)/f$ preserves value only when the same transaction pays the holder of the shifted field the matching cash leg $q \cdot c$; a market quote crossing the seam has that pairing, because the shareholder is paid, but a stored strike has no holder being paid, so the theorem by itself licenses no strike shift in either class. What decides the default is what the strike already contains. An ordinary, anticipated dividend sits in the forward the strike was struck against, so the strike stands — shifting it would create option value equal to the cash the boundary has already paid the shareholder, one distribution counted twice in the system's value accounting. A special dividend sits in no forward, so a standing strike would impose exactly that unpriced transfer in the opposite direction, and the strike moves to $(p - c)/f$ to hold the contracted position invariant. One principle yields both: a price terms field shifts by precisely the part of the distribution not already embedded in the reference it was struck against — zero for an ordinary dividend, all of c for every other class. The rule is read off that principle, not copied from a convention.
2. a **declared override** — a rule in a closed, first-order language that reads the boundary's parameters and the field's value and evaluates, with no recursion and no partial operations, to exactly one value. This is where a bespoke term sheet writes down its deviation.
3. **designated discretion** — the schedule names who owes a determination; until that party's attested *resolved terms* arrive referencing the boundary, the affected fields have no adjusted value, the boundary is pending for them, and any attempt to consume them is quarantined.

Adjusted terms enter the log from these two sources — the schedule's evaluator applying a declared entry, and the resolved terms of a designated party — and from nowhere else.

What makes this safe rather than merely organised is a totality gate. A registration, and every later terms append, is admitted only if the schedule yields exactly one well-formed entry for every pair of (corporate-action class, field) in the version being written; an append that introduces a new field must extend the schedule in the same transaction, or it is refused. The set of corporate-action classes is closed and finite — eight members: split, ordinary dividend, special dividend, spin-off, stock merger, elective merger, termination, and rebalance — so the gate is a finite check, and it is strict. An ill-formed override is refused, never quietly defaulted. The consequence is a strong one: past the moment a unit is registered, “the framework does

not know how to handle this event for this product” is not a state the system can reach. Either every class $\times$ field pair has an answer, or the version was never admitted.

The eight are closed by design, and closed does not mean incomplete: an event that looks like a ninth usually decomposes onto a member. A rights issue is the standing example. It is a spin-off — the right is a tradable series distributed to holders and registered as the child unit — not a new class of its own. A notice that genuinely falls outside the eight is refused, fail-closed; the taxonomy is widened only by changing this specification, never by an incoming datum.

A word on where the schedule is evaluated, because it decides what the door can and cannot catch. The schedule’s language is one definition, fixed once here, but it is executed at three distinct sites. The door executes it to check *totality* — that exactly one well-formed entry exists for every class $\times$ field pair — and nothing more; it never evaluates an entry down to a resolved value. The corporate-action processor executes it to *produce* the resolved terms at a boundary. And an independent recomputation, run later against the committed log, executes it a third time to *verify* those terms. The door’s evaluation proves an answer exists and is unique; it does not compute the answer, and it does not check the number the processor appended. That is precisely why a processor that appends a wrong resolved value — a perfectly well-formed terms version carrying the wrong strike — passes the door and is caught only afterward, by the recomputation re-running the same evaluation and comparing.

3.4 The datum-kind registry

Observations from the outside — a settlement price, a dividend forecast, a barrier fixing — enter the log as *stamped observations*. But not any observation may enter. Each *kind* of consumable datum is registered once, before any datum of that kind can be recorded, and the registration fixes the kind’s identity and declares a dimension for each of its components. A datum is an observation, never a holding, so its components draw on three dimensions — price, cash, dimensionless — but never quantity.

The reason a datum is registered per component, rather than named as a single number, is that one datum’s components can have different fates under the same operator. A dividend forecast is a good example. Suppose a forecast lists two components. The first is an amount of €3.00 per share; being quoted in the share’s price dimension, it takes the price action under a split, and it is declared in the *price* dimension. The second is a proportional part, 1% of price, declared *dimensionless*. A two-for-one split applies $f = 2$ to the whole datum, but the two components react by their own dimensions: the per-share amount halves to €1.50, the 1% stands. And both contributions are invariant, which is the point — $1,000 \times 3 = 2,000 \times 1.50 = €3,000$ on the cash component, and $1\% \times 100 \times 1,000 = 1\% \times 50 \times 2,000 = €1,000$ on the proportional one. A forecast settles nothing; it feeds a valuation. Had the proportional component been misdeclared as a price, the split would have halved it to 0.5% and contributed €500 to the mark where €1,000 is correct — half the datum’s contribution, from one wrong word in a declaration. This is exactly why each registry entry must carry a mandatory invariance witness, and it is worth saying precisely what the witness checks. It takes each component’s declared dimension and a scale boundary (f, c) and tests the value-invariance identity for that dimension: the quantity the component delivers must be the same before and after the boundary once the component is carried across by the map its declared dimension fixes. A component declared per-share in the price dimension satisfies $1,000 \times p = 2,000 \times (p/2)$ under a two-for-one; a component declared as an absolute cash amount is left unscaled and satisfies the identity only if it truly is a flat amount that does not depend on the share count. A per-share amount wrongly declared as absolute cash — or a flat amount wrongly declared as per-share — fails its identity under the replayed boundary, and the entry is refused at registration. The declaration is a trust assumption, and the witness bounds that trust to precisely its own width: a dimension that would misbehave under a scale cannot be registered in the first place.

A kind that was never registered is unconsumable: the door refuses it outright, as an

unregistered kind, returning the payload with the refusal but admitting nothing. This is distinct from a registered kind that a particular boundary’s declaration simply does not mention — that datum enters the log and is stopped only when it tries to cross that boundary. A registry entry, like a terms version, is immutable; a wrong declaration is corrected by registering a new kind and retiring the old one to new ingestion, never by editing the entry in place.

3.5 Operators: one declaration, two consumers

The last piece of declared data is the operator that carries a number across a boundary. When a corporate action is recorded, its declaration is a finite map from datum kinds to *operators*, drawn from a closed menu:

```
data OpSpec
  = Scale      Rational -- multiply a quantity/price by a nonzero factor
  | Shift      Rational -- subtract a per-share cash amount from a price
  | Subst      UnitId Rational -- succeed into another unit at a declared ratio
  | Recompose Basket -- resolve onto a basket of successor units
  | AId        -- identity, declared -- never assumed
  | Pending    -- boundary known, parameter not yet published
  | Terminal   -- the series ends; no conversion either way
```

The menu is closed: a declaration naming anything outside it is refused at the door, and widening the menu is a change to this specification, not something data can do. The menu is closed as an alphabet, not as a grammar: a declaration may bind a datum kind to a composite of menu operators — a succession at a declared ratio with a per-share cash leg is $\text{Subst}(Q, f) \circ \text{Shift}(-c)$, the interpretation of the single declared pair (f, c) as the price action $p \mapsto (p - c)/f$ — and closure means no operator outside the menu, never that members cannot compose. Two features of it repay attention. First, identity is a declared member, `AId`, and is never assumed. A declaration that omits a datum kind does not pass that kind through unchanged; it provides no conversion for it, and the door fails closed. The two-case function is exactly this:

```
convert :: Maybe OpSpec -> Datum -> Either Refusal Datum
convert Nothing _ = Left NoDeclaration -- omission is refusal
convert (Just op) d = apply op d -- AId is a 'Just', and is identity
```

The difference between `Nothing` and `Just AId` is the difference between “no answer, stop” and “the declared answer is: do nothing”. Only the second lets a datum cross. Second, the declaration is an operator, not a name. A name like “two-for-one split” would have to be interpreted, identically, by every consumer forever; an operator like `Scale 2` is simply applied. This is what lets one declaration serve two consumers.

A boundary’s declared operators are, in fact, consumed twice. They are consumed once at the boundary’s effective time, by the corporate-action contract that produces the boundary’s one-time transaction; and then they stand forever after at the read seam, converting every pre-boundary datum that is later read across the boundary. An example makes the two uses concrete. This read-seam application is the one generic computation the thesis of Section 1 named. Consider a single day. The log already holds May’s €2 ordinary dividend, recorded as `Shift -2`. At 09:59 a vendor prints a price of 100, stamped in the basis current at that moment — the pre-split frame, which the 10:00 split will leave one boundary back. At 10:00 a two-for-one split takes effect, and the corporate-action processor consumes the declaration `Scale 2` *once*: it moves 1,000 shares to 2,000, sets the new basis, appends the strike from €100 to €50 and the stored reference spot from €100 to €50, each tagged with the boundary. At 10:03 someone reads the 09:59 print in the new frame, and now the same `Scale 2` is consumed a *second* time, at the read seam: the stale 100 is converted to $100/2 = 50$, and only then is it allowed to meet the 2,000-share count. The ledger did not price anything and did not interpret the split; it applied the declared operator, and that is the whole of the generic work it does.

The composition of operators across several boundaries is a computation, not a choice. A May print of 102 read across both the dividend and the split composes in effective order: $(102 - 2)/2 = 50$. The reverse order, $102/2 - 2 = 49$, is not a valid conversion: no pair of bases stands in that relation, and the declared data offers no way to assemble it, so the arrival order of a datum can never change the number it converts to. Between any two bases on one chain there is at most one conversion per datum kind, and it exists exactly where every intervening declared conversion does. The €50 is not the ledger's opinion. It is the only number the declared data admits.

4 The Smart Contracts and Their Triggers

We have the door, the log, and the declared data a contract may read. What remains is the contract itself: the thing that, when an event fires, works out what must change and submits it. This section specifies what a contract is allowed to be, shows that trades, lifecycle events, and corporate actions are one kind of machine rather than three, and makes the two-layer correctness statement precise on worked numbers.

4.1 One architecture, three triggers

To the ledger, executing a trade and processing a lifecycle event are the same operation: a finite list of atomic moves with optional state changes, obeying conservation, admitted at the one door. There is no separate lifecycle subsystem inside the ledger, no per-product pipeline, no special data path for corporate actions. This is not an incidental economy; it is a direct consequence of the thesis. The ledger is a door, some checks, a log, and views, and nothing that admits or records a transaction cares where the transaction came from. So everything that changes the ledger is a transaction at the same door, and everything that produces a transaction presents the ledger with the same shape: a proposal to be admitted or refused. The producers' internals differ; the shape they present does not.

That shape has three trigger classes, and they differ in what they read, not in how they run:

- A **trade** is a contract triggered by execution. The executed trade becomes one conserving transaction; for a bespoke instrument the executed trade is itself the unit definition, registered at the moment it is done.
- A **lifecycle event** is a contract triggered by *the engine* — the external execution service that fires dated obligations — when an obligation comes due, an expiry, a coupon, an exercise. It reads the unit's current terms and state and the observations stamped for it, and it emits the moves that discharge the event.
- A **corporate action** is a contract triggered by a confirmed notice — a dividend, a split, a spin-off. It reads the notice's declared parameters and the unit's schedule, and it submits the boundary's one transaction, which may carry all three of a new terms version, the entitlement moves, and the status writes that set the new basis.

Each trigger fires a contract that produces exactly one transaction per firing. That one transaction may carry many moves, a terms version, and several status writes at once, but it is one atomic commit; a follow-on that a boundary sets in motion — a cascade onto a successor unit, a downstream settlement — is a separate trigger with its own transaction, not a second output of the first.

An option expiry and a stock split are processed by the same shape of machine: catch the trigger, read declared data and stamped observations, compute, submit. The differences between them are entirely differences in what they read. Nothing in the ledger branches on which of the three a given transaction came from.

The corporate-action trigger is a *confirmed* notice, and the gap between an announcement and its confirmation is handled deliberately. An announcement enters as declared data — the boundary is known, its effective time recorded — but it arms no operator: until confirmation, no conversion is declared, and nothing about any datum adjusts. The unit is marked pending for that boundary. Consumption that would cross the announced boundary fails closed and quarantines, because there is no operator to carry a number across it; consumption that stays in the unit’s standing basis — a fresh print taken before the effective time and read in the frame it was taken — is unaffected and proceeds normally. Confirmation is precisely the act that arms the operator, turning the announced boundary from a known-but-inert marker into a declared conversion the read seam can apply.

A single door settles quietly what happens when two things land on one unit at one instant, because there is no such instant in the log: every commit is totally ordered by the door, so events that share an effective time are simply adjacent, in a definite order. Two cases are worth stating. When two *boundaries* carry the same effective time on the same unit, the same-effective-time weld applies and effective order keys on the triple (effective time, declared precedence, bid); if the two notices declare no mutual precedence, the second is refused and the unit enters a pending-transition state — fail-closed — rather than letting the bid tie-break silently choose an order for two adjustments that do not commute. When a *lifecycle event* and a *boundary* fall together, there is no dedicated priority scheme, and none is needed: log commit order fixes which prefix each processor reads, prefix-determinism makes each submission a function of that prefix, and a settlement booked against a prefix that a boundary later precedes was correct on the prefix it read: recomputation reproduces it, diffs to zero, and does not flag it. What the retro-effective path guarantees is that the invalidation is exactly locatable and repairable, not that it is automatically found. The boundary’s window $[t_{\text{eff}}, t_{\text{notify}})$ is explicit in the log; replaying the corrected chain recomputes what each settlement in the window should have been; and the repair is a compensating transaction — authored outside the ledger, like every repair, admitted with lineage to both the settlement it corrects and the boundary that invalidated it. The guarantee is order, determinism, and repairability — not a ranking of event types.

A contract, then, is a deterministic, total function from its inputs to one transaction — a finite list of moves together with the terms versions and status writes that ride with them — with no randomness, no hidden state, and no dependency on anything not passed to it. Its output alphabet is exactly three things: moves, which route units between wallets and never create or destroy them; terms versions, appended never edited; and status writes, each an absolute replacement through its one legal writer. Conservation holds by construction over the moves, and the terms versions and status writes pass through the same admission checks as any other transaction. This is also why writing a contract from a legal confirmation is transcription rather than engineering. The confirmation already says what happens on each date and in each state; the contract is its machine-readable form, adding executability and adding nothing to the information.

4.2 What the contract layer is required to satisfy

Because a contract is untrusted, “deterministic and total” has to be made precise enough to check. Four obligations do that, and every implementation of a contract — the reference one, a vendor’s system, a rewrite — is held to exactly them.

- **Closed inputs.** A contract reads the committed log and the stamped observations the log records — nothing else. No ambient clock, no direct feed, no configuration off to the side. The timer that wakes a lifecycle contract is delivery, not data: it says *when*, while everything the event *means* is in the log. An expiry contract woken an hour late submits the identical transaction, because nothing it reads changed in that hour. The engine that fires these timers lives outside the trust boundary, named apart from the door, the log, and

the views, and holding no privileged access of any kind; it is bound only to fire durably and to deliver at least once. Because its trigger is delivery and never data, an engine that stalls, fails over, fires early, or fires twice can *delay* a discharge but cannot corrupt the log: a late delivery submits the same transaction, a duplicate delivery is absorbed idempotently at the door, and an early or spurious fire finds its observation or due-state not yet in the log and yields a refusal — a rejected proposal, never a corrupt commit.

- **Prefix-determinism.** Given a log prefix, the submission is determined. Two runs, or two independent implementations, over the same prefix produce equal transactions — equal move for move, under a *canonical move order*, the deterministic total order on a transaction’s moves fixed by the transaction itself, against which two recomputations are compared — so equality is decided rather than judged.
- **Idempotence keys.** Every submission carries a key derived deterministically from its cause: the obligation’s identity for a lifecycle settlement, the causing boundary paired with the target unit for a corporate-action follow-on. A retried submission is bit-identical, carries the same transaction id, and the door commits it once. Crash-and-retry cannot double-book.
- **Proposed, never authoritative.** A submission becomes ledger fact only by passing the same admission checks as every other transaction. No contract holds write privilege; no submission channel bypasses the door.

The reference implementation of a contract is a pure function — roughly,

```
handle :: LogPrefix -> Trigger -> Either Refusal Transaction
```

— and the four obligations are exactly the properties this signature already enforces: it takes the log prefix and the trigger and returns a transaction, with no argument through which a clock, a feed, or a configuration could enter, and no effect it could have except returning the transaction it proposes. Any other implementation is conforming precisely insofar as it matches what this function does on every prefix.

These four obligations make a contract’s *output* checkable, but they say nothing about whether an obligation fires at all. That is a separate guarantee, and the design carries it as data rather than as hope. Every contractual obligation is a first-class logged object, registered atomically with the position it binds and carrying three things: a deadline, a discharge predicate, and a compensation action. By its deadline it must reach one of three terminal states — Discharged, Compensated, or Defaulted — so a coupon that should fire and does not is not silence but a *Pending-past-deadline* object, visible to any fold of the log and therefore a detectable state rather than a missing one. The guarantee is conditional, and its primary condition is named: it rests on the engine being available at the deadline to deliver the trigger; Section 14 states the full set of conditions the guarantee stands on. Under an engine outage the obligation is reached *eventually*, when delivery resumes, not instantaneously; the log never loses the obligation, it only waits for the fire. Section 14 develops the obligation object and its liveness guarantee, including which component drives it to a terminal state.

4.3 Two layers of correctness, on numbers

We can now discharge the claim from Section 1 that a contract’s output is trusted for nothing and verifiable in everything. The subject is the contract, not the ledger: the ledger is precisely the trusted component — it is trusted to admit only well-formed transactions and to record them faithfully — and it is the contracts whose output is trusted for nothing and checked against the log instead. The claim splits into two layers, and the honest content of the split is that one layer is guaranteed and the other is checked.

The first layer is structural, and it holds against a defective contract and a malicious one alike, because it never depended on the contract. Whatever a contract submits, if the door

admits it then it conserves every unit, applies in full or not at all, writes every fact through its one legal writer, and dangles no reference. A contract cannot break this by being wrong, because being wrong is not one of the ways a transaction can be malformed. There is simply no unconserved, half-applied, or dangling proposal for the door to let through.

The second layer is economic, and this is the layer a contract can violate. Consider a settlement whose declared terms owe €100 per note, and a defective contract that submits a settlement paying €98. That transaction is balanced, conserving, and basis-consistent. The door has no ground to refuse it — every structural property holds — and it is admitted. On a 1,000-note holding it has now booked €98,000 where €100,000 was owed. Nothing at the door catches this, and nothing at the door is supposed to. What catches it is recomputation: the settlement is recomputed from the log prefix — the terms, the schedule, the registered kinds, the stamped observations are all there — and compared with what was recorded, in canonical form. The comparison reports a difference of €2 per note, €2,000 on the holding. That nonzero difference is the defect, located exactly, and it is a defect in the contract, not in the ledger. It is repaired downstream by a compensating transaction — authored outside the ledger, by the corrected producer, and resubmitted through the same door under a `corrects` field that names the €98 settlement it supersedes. The ledger admits this repair exactly as it admits any transaction and manufactures none of it; the original €98 settlement stays in the log, marked corrected, so the error and its repair both survive.

The corporate-action half of the system runs the same check on the same kind of numbers. A declared `Scale 2`, composed with the schedule, takes a €100 strike to €50. A defective contract that appends €49 instead has submitted something structurally admissible — a well-formed terms version — and it is caught the same way, by recomputing the appended strike from the boundary and the schedule and finding the €1 gap. One architecture, one verification mechanism, on both halves: a settlement that should pay €100 and an adjustment that should yield €50 are checked by the same recomputation, and a wrong €98 or a wrong €49 is found the same way.

This is the whole of what Principle 1.1 claims, now on numbers. The ledger *relies on* the structural layer, because it is guaranteed by construction and no contract can break it. It *expects* the economic layer, and rather than trusting the expectation it makes it checkable: “the contract computed the right number” is not believed, it is recomputed. A failed recomputation is always a producer defect and never a ledger defect, because the ledger’s part — admitting only well-formed transactions and recording them faithfully — is exactly the part that was guaranteed.

The component that performs this recomputation deserves a name, because it, and not the door, is what makes economic correctness checkable. Call it the *witness*. The witness takes the committed log, replays the declared inputs a transaction depended on — the terms, the schedule, the registered kinds, the stamped observations — recomputes the transaction in canonical form, and compares. Every input it reads is log-resident and its computation is deterministic, so any independent party can run it and reach the identical verdict; who operates it — the firm, a counterparty, an auditor, a regulator — is a deployment choice, not a matter of trust. The witness is not part of the door. The door admits well-formed transactions in log order and moves on; the witness reads the committed log after the fact. Nothing gates admission, a view, or the settlement projection on a prior witness pass, and when the witness runs is an operational decision the deployment must schedule. This is why the bound is eventually-detected-and-repaired rather than verified-before-consumed: the check is real and anyone can run it, but it runs over what is already committed, not before it.

4.4 Provenance is recorded, never acted on

Every transaction that follows from a boundary carries a tag naming its cause — the boundary that triggered it. It is tempting to think the ledger uses this tag to decide how to process the

follow-on. It does not. The tag is provenance only. A unit's own boundary transaction carries no cause tag, because a boundary is its own cause: its identity is the transaction's own entry in the log, and a tag pointing a boundary at itself would be redundant at best and circular at worst. Every follow-on and cascade, by contrast, carries the causing boundary's identity. But no admission check, no valuation, no conversion, and no code path anywhere branches on the tag's value. It is read only by the witness that verifies economic correctness, and by audit.

The one gate that does look at the tag is a well-formedness check, not a behavioural branch: a transaction that names a cause is admitted only if that cause is a boundary already committed strictly earlier in the log. This refuses a transaction that points at an event not yet recorded, and it structurally rules out a transaction that names itself. The tag names the cause; it does not carry the conversion. Audit needs the name, arithmetic needs the operator, and the two are kept apart on purpose: the tag alone is necessary for provenance and insufficient for computation, which is exactly why nothing computes from it.

4.5 The model-agnostic wall

One boundary has been implicit throughout and is worth making explicit, because it is what keeps the whole architecture indifferent to product complexity. The ledger records the outputs of pricing models and never runs a pricing model. Whatever computation decided that a structured note was worth what it was worth — however elaborate, however proprietary — is not something the ledger reproduces, validates, or even sees. What the ledger sees is the moves the contract emits and the observations stamped into the log. The repricing an event causes is new information, and it enters only as a fresh stamped observation; the ledger never manufactures it by transforming an old observation into a new one. The gap between a transported old price and the next fresh print is market profit and loss, and it is left as such, never absorbed into a boundary.

This is why the four obligations above quantify over the log prefix and the stamped observations and mention no model. A pricing function is a black box on the far side of the wall. A declared operator — the one generic conversion of Section 3 — is a known, closed-menu transformation on the near side. The wall between them is what lets the ledger process a bespoke note and a plain share with the same machine: it never has to understand either one, only to admit the conserving transactions their contracts propose and to recompute those transactions from data that is entirely in the log.

5 The Product Tour

The three sections before this one built one machine: a single door, a handful of checks, an append-only log, and views computed from it, fed by contracts that read only declared data and stamped observations. The claim was that this one machine carries every post-trade product, not a family of machines each shaped to its asset class. A claim like that is not settled by argument. It is settled by running real products through the machine and watching it hold.

That is what the next eight sections do. Each takes one product family, picks the event on it that is hardest for the design, and works that event through in full. The stops are ordered by machinery, not by asset class: the trade and the split first, because they are the plainest triggers; then two lifecycle products, a listed future and a structured note; then corporate actions in ascending order of bespokeness, from a special-dividend override to a spin-off to an elective merger; and securities lending last, because it is the one stop that changes what a position *is*, and so tests whether the architecture survives that change.

Every stop keeps the same five-part shape, so that what differs between products is visible against what does not.

- **The trigger.** What fires — an executed trade, a dated obligation coming due, a confirmed corporate-action notice.
- **The contract that catches it.** Which of the three trigger classes runs, and what declared data and stamped observations it reads.
- **The transactions it produces.** The moves and status writes it submits, shown as numbered rows with their exact quantities.
- **What the door checks.** Which admission check is load-bearing here, and what it refuses.
- **The design choice it substantiates.** One sentence, at the close, naming the single decision from the mechanism sections this product would break without.

The stops share one setting — a single firm’s book over one year — but each is a self-contained worked example, and each opens with its own stated starting position rather than inheriting a share count from the stop before. A recurring name, ACME, carries the narrative: the same firm trades ACME, sees it split two-for-one, receives a dividend forecast on it, holds an option over it through a September special dividend, watches it spin off a subsidiary, and the following March sees it acquired in an elective merger; the securities-lending stop works that same September through a book that has lent part of its ACME. The tour makes no claim that one exact holding is carried unchanged from stop to stop — each stop states the position it works with, so that what the machine does to that position is visible without the reader tracking a running balance across a year. Two stops run their own lines, because the architecture is not equity-shaped and the tour should not pretend it is: a dollar-denominated listed future, and a structured note whose pricing the ledger never sees. Every number within a stop is exact, and every one is a number the log would produce for the position that stop states.

6 Cash Equity and the Split Phantom

The plainest product makes the plainest case, so the tour opens on cash equity: a purchase, its cash dividend, and then the split that is the first event with a way to go quietly wrong.

6.1 The trade and its dividend

The trade is where the ACME line begins. The book buys 1,000 ACME shares at €100, and the executed trade is a contract triggered by execution: it produces one transaction, and that one transaction, at trade date, books both legs at once. It brings 1,000 of the ACME share unit from the counterparty’s virtual wallet into the book’s wallet, and it moves €100,000 of cash the other way, into the counterparty’s virtual wallet — whose balance is now the unsettled payable the book owes. Both legs are booked the instant the trade is struck, because on the trade date the position is already owned and the cash is already due; what has not yet happened is the external transfer that will hand the counterparty its money. The door checks that ACME is a registered unit and that the transaction is well-formed, appends it, and the book’s share balance is 1,000 — not a stored position some later process must maintain, but the sum of the one move the log records — while the counterparty’s virtual cash balance is the €100,000 still owed. Settlement, when it comes, generates no moves at all: the positions were already correct from trade date, and the external transfer of the cash is recorded as state — the payable’s confirmation, marked on the transaction the log already holds — not as a second transaction. The trade-date booking and the settlement-date transfer are two dated facts joined by that virtual-wallet balance, not a single instant in which shares and cash change hands together; conservation holds within the one trade-date transaction, and nothing new is needed to express it.

The cash dividend is the simplest lifecycle contract there is, and it is worth seeing in full precisely because there is so little to it. ACME declares a €2 ordinary dividend. This is the boundary $(f, c) = (1, 2)$: the quantity factor is 1, so no share count changes, and the per-share cash is €2. The contract, triggered by the confirmed notice, reads the terms and the holding and submits one transaction. It pays cash $1,000 \times 2 = €2,000$ into the book's wallet, and it records the declared operator **Shift** -2 as the boundary the read seam will thereafter apply to prints: a cum-dividend quote of €102, read across that boundary, transports to $(102 - 2)/1 = €100$, exactly the price going ex-dividend. The stored price reference on the instrument itself is not moved — under an ordinary dividend a price terms field stands, and it is a market print crossing the seam, not a stored field, that the boundary shifts. Value is conserved across the boundary up to the cash the boundary paid: €102,000 of shares beforehand is €100,000 of shares plus €2,000 of cash after. There is no separate dividend engine. There is a contract that reads a boundary and emits moves, and it is the same shape of contract that will, in a moment, process a split.

A word on the feeds, because this basis convention is the ledger's own. Exchanges and vendors rebase a quote series for a split or a large special dividend, but not for an ordinary one: a print taken the morning after an ordinary ex-date arrives *unadjusted*, in the same basis as the day before. Treating the €2 ordinary dividend as a boundary that carries a pre-ex quote to its ex-frame is therefore a convention the ledger declares and applies at the read seam, reconciled there against feeds that do not share it — not a frame the feed already speaks. Nor does the convention forecast the market: the realised ex-drop is nearer $c(1 - \tau)$ than the full c , because the dividend is taxed in the holder's hands, and that gap between the transported quote and the next fresh print is ordinary marking profit and loss, never something the boundary absorbs.

6.2 The split, and the phantom it invites

On the first of June, ACME splits two-for-one. The corporate-action contract reads the confirmed notice, whose declaration is **Scale** 2 — the boundary $(f, c) = (2, 0)$ — and submits one transaction that does three things at once: it moves the holding from 1,000 to 2,000 shares, it appends a new terms version carrying the halved reference price of €50, and it writes the new basis. Because it is one transaction, no reader can ever see the shares doubled while the basis still says pre-split, or the basis advanced while the shares have not moved. The door's invariance weld admits it only if the quantity leg realises $q \mapsto q \cdot f$ with no cash leg, which **Scale** 2 with $c = 0$ does.

Now the event that breaks careless systems. At 09:59, one minute before the split takes effect, a vendor prints an ACME price of €100, stamped in the pre-split basis. At 10:03 someone values the position. The share count is now 2,000. The most recent print is €100. Multiply them and the book is worth $2,000 \times 100 = €200,000$. Every factor here is individually correct: 2,000 is the true share count, and €100 was a true, recently printed price. The product of two correct numbers, €200,000, contains €100,000 of phantom value on top of the true €100,000 — and no internal reconciliation can catch it, because nothing internal disagrees: the position system and the valuation system are both reading real data. The error is that the two numbers live in different frames: the quantity is post-split, the price is pre-split, and they were never entitled to meet.

The right number is €100,000, and it can be reached from either frame. In the pre-split frame, $1,000 \times 100 = €100,000$. In the post-split frame, the stale print is first transported across the boundary by the same declared **Scale** 2, now consumed at the read seam: $100/2 = €50$, and only then allowed to meet the 2,000-share count, $2,000 \times 50 = €100,000$. Both frames agree because there is one declared operator and it is applied wherever a pre-split number tries to cross. Single-basis consumption is the rule the door enforces here: a quantity and a price may be multiplied only in one frame, and it is the ledger that decides the frame, from the declared boundary, not the feed that happens to have printed last.

The design forbids the phantom by refusing to build it. The 09:59 print is not deleted or corrected — it is real data and stays in the log — but the pairing of a pre-split price with a post-split quantity is not a wrong number the door computes and lets through. It is a transaction that does not typecheck, and a stale datum met at a boundary with no declared conversion is a typed refusal, not a silent €200,000.

One more turn shows why the frame has to be decided by declared order and not by arrival order. Suppose the book pulls a history a week later, and reads a May print of €102 across both the May dividend and the June split. The two boundaries compose in effective order: dividend first, then split, giving $(102 - 2)/2 = €50$. The reverse order, $102/2 - 2 = €49$, is a different number, and it is not a valid conversion of anything: no pair of bases on the chain has it, and no operator the menu can build will forge it. The arithmetic does not commute, so if arrival order could pick the order of composition, the same datum would convert to €50 or €49 depending on when it happened to be read. It cannot, because the composition is fixed by the declared boundaries, and €50 is the only number the declared data admits.

None of this required restating a single stored share. The 1,000 became 2,000 by one recorded move, and the reference price became €50 by one appended terms version; the old count and the old price both still stand in the log exactly as written. *The decision this stop rests on is single-basis consumption composed in declared order: a quantity and a price meet only in one frame, the ledger fixes that frame from the declared boundary rather than from whichever feed printed last, and when boundaries stack it is the declared effective order — not the order a datum happens to arrive — that fixes the single number it converts to, which is why €50 is forced and €49 cannot be assembled. That quantities are never restated in place is the visible consequence, not the cause: the phantom is unrepresentable because the pairing that would build it does not typecheck, not because a restatement was performed carefully.*

7 A Dividend Forecast as a Two-Component Datum

The split stop showed a single number — a price — crossing a boundary. This stop shows why the ledger insists that an observation declare a dimension for each of its parts, rather than arriving as one number under one name. A dividend forecast on ACME is the example, because its two parts react differently to the very same split.

7.1 One datum, two components

That same June, a data authority stamps a dividend forecast on ACME. It is not one number. It has two components, and the datum-kind registry required both to be declared, with a dimension each, before any forecast of this kind could enter the log:

- a **cash-per-share** component of €3.00, declared with dimension *price* — it is quoted in the share’s own price basis;
- a **proportional** component of 1% of price, declared *dimensionless* — it is a ratio, carrying no basis of its own.

A forecast settles nothing — it is an estimate, and cash pays only off the declared dividend terms at the ex and pay dates. What a forecast feeds is a *valuation*: a dividend-swap or forward mark reads a theoretical figure off it. Against a holding of 1,000 shares at €100, the two components contribute $1,000 \times 3 = €3,000$ from the cash part and $1\% \times 100 \times 1,000 = €1,000$ from the proportional part to that mark. So far nothing forces the components apart. The split does.

The two-for-one split declares **Scale 2**, and the read seam applies it to the forecast when the forecast is next read in the post-split frame. But it applies to each component by that

component’s own declared dimension, not to the datum as a lump. The cash-per-share component is a price, so it maps $p \mapsto (p - c)/f$ with $c = 0$: €3.00 halves to €1.50. The proportional component is dimensionless, so it stands: 1% stays 1%. And both contributions are exactly invariant, which is the whole point of splitting the datum in two:

$$1,000 \times 3 = 2,000 \times 1.50 = \text{€}3,000, \quad 1\% \times 100 \times 1,000 = 1\% \times 50 \times 2,000 = \text{€}1,000.$$

The dividend value the forecast carries did not change when ACME split; only its per-share expression did, and it changed correctly because each component reacted by the dimension it was declared under.

7.2 One wrong word, half the money

Now suppose the proportional component had been declared wrong — registered as a *price* instead of dimensionless. Under **Scale 2** a price halves, so the 1% would be taken to 0.5%, and the component would contribute $0.5\% \times 50 \times 2,000 = \text{€}500$ to the mark where the correct figure is €1,000. Nothing settles off this — a forecast pays no cash — so the damage is not a wrong payment but a corrupted *valuation*: the position would be marked €500 light on that leg against the correct €1,000, half the datum’s contribution lost to a single wrong word in a declaration. And nothing downstream would look wrong, because a mark is a well-formed number that no conservation check disputes. This is why every registry entry must carry an invariance witness: a check that replays a scale boundary against the declared dimensions and refuses the entry if a component’s declared dimension does not survive it. The dividend-forecast kind carries exactly such a witness, and it fails the mis-declared variant at registration, before any forecast of that kind can be stamped — before the mis-typed component can enter any view. The declaration is a trust assumption, and the witness checks exactly what it can: dimensional consistency — that each component’s declared dimension satisfies the invariance identity under a replayed scale — not whether the forecast’s numbers are true. Whether €3.00 is the right estimate is not the witness’s question, and it is never claimed to be.

A forecast is a model’s output — some team’s estimate of a future payment. What matters for the ledger is that the estimate does not cross the door as an authoritative number to be interpreted. It crosses as a stamped observation of a *registered kind*, whose components carry declared dimensions the ledger already knows how to transport. A kind that was never registered is simply unconsumable: the door refuses it, returns the payload with the refusal, and admits nothing. *A forecast enters as a stamped observation of a registered kind with a dimension per component, never as a model number the ledger must trust; an unregistered kind cannot be consumed at all, and a mis-declared one is caught by the kind’s own witness before it can mark the position half light.*

8 Listed Futures Daily Settlement

A listed future behaves unlike the others: it settles every day, it posts variation margin, and desks run whole engines to manage that margin. This stop shows that daily settlement needs no engine of its own. It is a scheduled trigger producing ordinary conserving moves, and the margin that looks like special machinery is just those moves summing to zero.

The contract is a real one, and it is dollar-denominated, so the tour states it as such rather than forcing it into euros: ES-FUT, a cash-settled equity-index future, contract multiplier 50, quoted in USD. Three traders, A, B, and C, and a clearinghouse hub CH stand behind them. Two rules run the whole life:

- **A trade of Δ contracts at price p** adds $-\Delta \cdot p \cdot 50$ to the accumulated cost ac of each side.

- **A settle at price S** pays each holder variation margin $VM(w) = netq(w) \cdot S \cdot 50 + ac(w)$, then resets $ac(w) \leftarrow -netq(w) \cdot S \cdot 50$.

The daily settle mark S is not a number the ledger forms. It enters as an externally sourced observation carrying its basis, and it is consumed under single-basis consumption: the mark and the positions it is multiplied against must stand in one frame, exactly as the split stop required.

8.1 The life of the contract

Table 1 is the whole life, in position ($netq$), accumulated cost (ac), and the variation-margin cash each settle pays, for (A, B, C).

Step	Event	$netq$	ac (USD)	VM cash (USD)
T1	A buys 10 from B @100	(10, -10, 0)	(-50,000, +50,000, 0)	—
Settle d1	$S_1 = 102$	(10, -10, 0)	(-51,000, +51,000, 0)	(+1,000, -1,000, 0)
T2	C buys 4 from A @103	(6, -10, 4)	(-30,400, +51,000, -20,600)	—
Settle d2	$S_2 = 101$	(6, -10, 4)	(-30,300, +50,500, -20,200)	(-100, +500, -400)
T3	B buys 4 from C @101	(6, -6, 0)	(-30,300, +30,300, 0)	—
Expiry	final $S = 105$	(6, -6, 0)	(-31,500, +31,500, 0)	(+1,200, -1,200, 0)

Table 1: ES-FUT over its life. Each settle is a scheduled trigger; variation margin is the cash the settle contract moves.

Two of the numbers repay a moment. Day 1 is the easy case: A is long 10 into a mark that rose from 100 to 102, and collects $10 \times 102 \times 50 - 50,000 = \text{USD } 1,000$, which B pays. Day 2 looks harder, because A traded intraday. A sold 4 contracts to C at 103 before the mark fell to 101, and A's day-2 margin is $6 \times 101 \times 50 - 30,400 = -\text{USD } 100$ — not the naive $6 \times (101 - 102) \times 50 = -300$ a per-position mark-to-mark would give. The difference is the USD 200 A realised selling 4 contracts at 103 into a prior mark of 102, and the accumulated cost carried it correctly, because ac is a stored, conserved field and not a re-derivation each day.

8.2 Margin is just conservation

The point of the table is the last column. At every settle the variation margin sums to zero across the holders: $+1,000 - 1,000 = 0$; $-100 + 500 - 400 = 0$; $+1,200 - 1,200 = 0$. This is not a reconciliation the ledger performs after the fact. It is the same fact as accumulated-cost conservation, $\sum_w \Delta ac = 0$, which holds because every settle is built from moves and a move is a transfer: whatever leaves one wallet enters another. The clearinghouse leg is the residual that balances the holders' legs, and here, with every holder inside the ledger, that residual is zero each day. Position conservation, $\sum netq = 0$, holds at every step for the same reason.

Because each settle is keyed by (unit, settlement date), a settle that fires twice — a processor retried after a crash, a timer that fired late — commits once. And the zero-sum closes over the whole life, not just each day. Cumulative margin is $A = +2,100$, $B = -1,700$, $C = -400$, and $2,100 - 1,700 - 400 = 0$: the contract's entire economic history is a single conserved stream, checkable end to end. A's USD 2,100 is exactly $[4 \times (103 - 100) + 6 \times (105 - 100)] \times 50$ — what A made on the 4 contracts sold at 103 and the 6 carried to expiry at 105 — so the running margin and the closed-form profit agree to the cent.

One margin lives outside this picture, and the stop should say so plainly. Initial margin — the portfolio or SPAN amount a clearinghouse holds against future moves — is gross, not zero-sum: every member posts it to the clearinghouse and no member's posting offsets another's, so it is not a conserved transfer among the holders and does not belong to the variation-margin stream this table records. It is a collateral position posted to the clearinghouse, booked like

any other collateral through the same door, and its sizing is a risk model’s output the ledger records but never forms. The zero-sum here is a statement about *variation* margin — the daily cash the mark moves between holders — and about nothing else.

The arithmetic never divides. A settle multiplies $netq \cdot S \cdot m$ and nothing else — prices are never added to prices, only multiplied by an integer position and an integer multiplier — so there is no place for a rounding error to enter and accumulate. The ES mark ticks in quarter-points, and a quarter-point on a multiplier of 50 is an exact USD 12.50, so even the finest price increment lands on a whole number of cents and the zero-sum holds to the cent at any scale. *Variation margin is ordinary conserved transactions keyed per settlement day — there is no privileged variation-margin engine inside the ledger; the same door and the same log that carry a share purchase carry a cleared future’s entire variation-margin life, while initial margin sits outside as a posted collateral position.*

9 A Structured Note, Model-Free

If any product should break a ledger that refuses to run a pricing model, it is a structured note: a coupon, a barrier, a payoff computed by proprietary machinery that takes a quant team to reproduce. This stop runs one through the machine and shows that the ledger never needs to open that machinery. The specific structure here — a periodic coupon with a knock-out barrier — is an illustrative composition of real, separately specified mechanisms; the note’s price is a black box, and no pricing formula appears anywhere in this stop, because none ever crosses the door. What crosses are contractual cash flows and stamped observations, and those are all the ledger ever needs.

9.1 The coupon: a scheduled cash move

The book holds 1,000 notes, quoted €102 cum-coupon, each paying a €2 coupon. A coupon date is a dated obligation, so it is a scheduled trigger, and the lifecycle contract that catches it reads the declared terms and the holding and submits one transaction. Economically the coupon is the ordinary-dividend boundary with $(f, c) = (1, 2)$: it books cash $1,000 \times 2 = €2,000$ and advances the quote from cum to ex, €102 to €100. Value is invariant across the date up to the cash paid: $1,000 \times 102 = €102,000$ beforehand is $1,000 \times 100 + €2,000$ after. Because the coupon and the ex-move ride in one atomic transaction, the two ways it could corrupt — cash paid while the quote stays cum, so the coupon is counted twice; or the quote gone ex with no cash, so the coupon is lost — are both unreachable. The contract records the coupon as paid in the unit’s status, so a re-fire on the same date pays it once.

9.2 The barrier: an observation, not an opinion

The note carries a knock-out barrier. Say, illustratively, the barrier sits at €80 on the underlying — a price-in-basis terms field — and on the fixing date the official close prints €78.40. The ledger does not decide whether a barrier “should” have breached, and it runs no model that would form such a view. The close enters through the ingest door as a stamped observation, and the contract reads it against the declared barrier level. If it knocks, the knock is first a recorded change of the unit’s lifecycle state — ACTIVE to TERMINATED. Whether that termination also owes cash is a separate matter of the terms: many knock-out structures pay a rebate or the accrued coupon at termination, and where they do, that leg is a dated settlement obligation, discharged on its own value date when it falls due, not conjured at the instant of the breach. The state-change transaction itself is still move-less: the breach is recorded now, and any cash it triggers settles later as its own obligation.

A move-less transaction may look like a transaction that does nothing, but it is load-bearing. Without the recorded state change, a later reconstruction of the ledger as of that date could not

tell “terminated by breach” from “still active, and merely held by no one” — two economically distinct states that would otherwise look identical in the balances. The state change is what the note’s changed liability consists of, so it is recorded even though no cash moved. Consumption of the fixing is legal only under single-basis consumption: the stamp’s basis must agree with the frame it is read into.

9.3 Expiry, and the two layers on one number

At expiry the note settles, and the settlement is where a wrong contract can do its worst — and where the ledger’s answer to that is exact. Suppose the declared terms owe €100 per note, and a defective contract submits a settlement paying €98.

The first thing to see is that the door admits it. That €98 transaction is balanced, conserving, and basis-consistent; every structural property the door checks holds. The door has no ground to refuse it, and it should not — refusing well-formed transactions on suspicion is exactly how a system of record starts improvising. So the book now carries $1,000 \times 98 = €98,000$ where €100,000 was owed. This is the structural layer doing its job: it guarantees the transaction cannot break the ledger, and it says nothing about whether the economics are right, because it never depended on the contract being right.

The second thing to see is that the error cannot hide. Everything the settlement was computed from — the terms, the adjustment schedule, the registered kinds, the stamped observations — is in the log. So the settlement is recomputed from the log prefix and compared with what was recorded, in canonical form. The recomputation yields €100, the record says €98, and the comparison reports a difference of €2 per note, €2,000 on the 1,000-note holding. That nonzero difference is the defect, located exactly, and it is a defect in the contract, not in the ledger — the ledger’s part, admitting only well-formed transactions and recording them faithfully, is exactly the part that was guaranteed. The repair is a later compensating transaction, itself recorded — and it is worth separating two things that are easy to run together. That the error cannot hide is guaranteed by the ledger: the €2 per note is in the log for anyone to recompute. That the €2,000 actually comes back is not a ledger operation at all — the €98,000 already left the door to an external holder, and recovering the shortfall is a claims process at the boundary, outside the system of record. The ledger makes the error undeniable and the correction recordable; it does not reach across the boundary and unwind a payment already made.

Nothing in this stop asked what the note was worth. The pricing machinery that decided the coupon, the barrier, and the settlement terms is on the far side of a wall the ledger never crosses: it records the moves the contract emits and the observations stamped into the log, and it recomputes those from the log, and that is all. Were an event to reprice the note, the reprice would enter only as a fresh stamped observation; the ledger never manufactures a new price by transforming an old one, and any gap between a transported old price and the next fresh print is left as market profit and loss, not absorbed into a boundary. *Lifecycle runs on declared terms and stamped observations alone — a pricing model’s output enters only as a stamped observation carrying its basis, never as an authoritative number the door interprets or trusts; the door catches the transactions that are broken, and recomputation from the log catches the ones that are merely wrong.*

10 A Bespoke Special-Dividend Override

Real term sheets deviate from the standard treatment, and a system that hard-codes the standard treatment has to be reprogrammed each time one does. This stop shows a deviation absorbed as data. In September, ACME pays a special dividend, and the book holds an option written over ACME whose confirmation adjusts the strike in a way the class default would get

wrong.

10.1 Where the default and the term sheet disagree

The option's strike is €100. ACME declares a special dividend of €2 per share, the boundary $(f, c) = (1, 2)$. The class default for a strike — a price-in-basis terms field — under a special dividend is the full adjustment $p \mapsto (p - c)/f$, which here is $K - c = 100 - 2 = €98$. This default is unconditional: once the event is classified special it applies in full, with no threshold and no proportioning. And the classification itself — ordinary or special — is the confirming authority's determination, recorded in the notice and made outside the ledger; there is no ledger-side test that inspects the dividend's size and decides the class. The ledger reads the class off the confirmed notice and applies the default that class carries.

It is worth seeing why that default is what it is, because the invariance arithmetic alone does not settle it. The re-expression $p \mapsto (p - c)/f$ preserves value only when the same transaction pays the holder of the shifted field the matching cash leg $q \cdot c$. A market quote crossing the ingest seam has that pairing — the shareholder who holds the quoted share is paid the dividend — but the holder of an option over the share is paid nothing. So Theorem 3.1 by itself licenses no strike shift in either dividend class: the cash leg that would balance a shift is simply not present for a strike. What decides the default is what the strike already contains. An ordinary, anticipated dividend already sits in the forward the strike was struck against, so the strike stands: shifting it down would hand the option holder per share exactly the cash the boundary has already paid the shareholder — one distribution counted twice in the system's value accounting. A special dividend sits in no forward, so a standing strike would impose that same unpriced transfer in the opposite direction, this time against the holder; the strike moves to $K - c$ to hold the holder's contracted position invariant. One principle yields both cases: a price-in-basis terms field shifts by precisely the part of the distribution not already embedded in the reference it was struck against — zero for an ordinary dividend, the whole of c for a special. On an ordinary €2 dividend the strike therefore stands at €100, even as the read seam carries a €102 market print across the same ex-boundary to €100: the quote moves because its holder was paid, the strike does not because the option holder was not. On the special €2 dividend the same principle moves the strike the full c , to €98.

That €98 is the default. This option's confirmation, though, says something narrower: the strike adjusts on a special dividend only to the extent the dividend exceeds €0.50, and only by the excess. That is not the class default and not a market norm — it is this term sheet's own *declared override*, written as a rule in the schedule's closed first-order language that reads the boundary's declared cash c and the field's value K and evaluates to

$$K \mapsto \begin{cases} K - (c - 0.50) & \text{if } c > 0.50, \\ K & \text{otherwise.} \end{cases}$$

On this €2 dividend it yields $100 - (2 - 0.50) = €98.50$ — the excess-only figure the confirmation requires, and the number the contract must produce. The rule is total, and its boundary behaviour is exact: at $c = €0.50$ the condition $c > 0.50$ is false, so the strike stays €100; one cent above, at $c = €0.51$, it moves by a single cent to $100 - 0.01 = €99.99$. There is no gap and no ambiguity at the threshold, because the rule evaluates to exactly one value for every c . The threshold and the excess-only treatment live entirely in this override; the class default behind it remains the full €98.

10.2 Three layers, and a gate that will not improvise

The override is one of three ways a schedule entry can be filled, and the three form a strict precedence, most specific winning:

1. the **class default**, fixed once in the specification per corporate-action class and dimension — what a strike does absent any deviation;
2. a **declared override**, a total rule in the closed language, reading the boundary’s parameters and the field’s value — where this term sheet’s €98.50 lives;
3. **designated discretion**, which names who owes a determination and leaves the field with no adjusted value until that party’s attested resolved terms arrive referencing the boundary; until then the boundary is pending for that field and any attempt to consume it is quarantined.

What makes this safe rather than merely tidy is a totality gate. A unit is registered, and every later terms version admitted, only if the schedule yields exactly one well-formed entry for every pair of corporate-action class and terms field. The set of classes is closed and finite, so the check is finite, and it is strict: an ill-formed override is refused at the gate — **ScheduleIncomplete** — never quietly defaulted to the class rule. The consequence is strong. Past the moment the option is registered, “the framework does not know how to adjust this field for this event” is not a state the system can reach. Either every class-and-field pair has exactly one answer, or the version was never admitted.

So the special dividend runs through the same corporate-action contract as an ordinary one. Nothing branches on the fact that this unit is bespoke. The contract reads the schedule, finds the override entry, and appends the €98.50 strike; recomputation from the logged notice re-runs the rule on ($K = 100$, $c = 2$, threshold 0.50) and confirms €98.50, diff zero. *Bespokeness is a higher-precedence layer of declared data behind a totality gate that fails closed — never a special case in code; the same contract that adjusts a plain strike adjusts a bespoke one, because the deviation was written as data, not as a branch.*

11 The Spin-Off

A spin-off moves value across instrument lines: one holding becomes two. This stop shows the ledger doing that in a single transaction, and — the harder part — being honest about a price it does not yet have. In November, ACME spins off a subsidiary. Write the parent as A and the two resulting lines as A_1 (the continuing ACME) and A_2 (the new company). The book holds 1,000 A , marked €100, so €100,000.

11.1 One transaction across two lines

The confirmed notice says each share of A entitles the holder to one A_1 and one A_2 . The corporate-action contract reads it and submits a single transaction that issues 1,000 A_1 and 1,000 A_2 into the book’s wallet against the 1,000 A it resolves — A_1 and A_2 each registered as units at their origins, with their own datum kinds. Because it is one transaction, the two new lines appear together or not at all; no reader can catch the book holding A_1 but not yet A_2 , and per-unit conservation holds on each line separately, each new share issued by a move from a virtual wallet. This is cross-instrument transport, and it is one atomic multi-unit transaction, not a sequence that some interleaving could observe half-done.

The transaction carries the entitlement — the quantities. What it cannot carry is the children’s prices, and this is the point of the stop. No notice can contain a price formed after it. In practice both lines often trade *when-issued* in the days before the distribution settles, so per-component prices may well exist ahead of the first regular-way session; but a when-issued print is market data, not terms, and the notice that issues the shares still does not contain it. Say the lines settle near $A_1 = €80$ and $A_2 = €20$ once they trade regular-way. Whether the continuing parent is modelled as a fresh unit A_1 at all is itself a choice — ACME’s listing and

ISIN usually survive the spin-off, so a deployment may keep the parent line and register only the new child; this stop registers both to work the general two-line case.

11.2 A mark carried only as far as the data supports

So on the morning after the spin-off, before the children have printed, what is the holding worth? The stale €100 parent mark still exists, stamped in the parent's basis. The declaration at the boundary is **Recompose** of A onto the basket $\{A_1 \mapsto 1, A_2 \mapsto 1\}$, and that basket datum was jointly stamped at the successors' origins. It is consumable, but only jointly. It licenses a mark on the *pair*: the combined 2,000 shares — 1,000 A_1 and 1,000 A_2 together — carry €100,000, which is exactly the pre-spin value, phantom-free, so the overnight risk of the combined position is readable at the open.

What it refuses is a mark on either child alone. To place a number on A_1 by itself, one would have to divide the €100,000 between the two lines in some proportion — and that proportion is exactly the decomposition the notice never contained. Its absence is what single-basis consumption refuses on: the per-component kinds stay pending until fresh per-component observations arrive to supply it. So the combined pair can be valued and its overnight risk read, but the two lines cannot yet be hedged or margined against each other, because separating their risk needs the very split that is still pending. That split is not refused forever, only until it is observed: the issuer's cost-basis allocation, or the first regular-way session's volume-weighted prices, later supplies it — the allocation as attested declared data, the prices as stamped observations of their own. The basket step is a bridge, not a standing conversion; it retires the moment the children carry their own fresh stamps, after which valuation proceeds component by component.

The discipline earns its keep against a tempting error. Suppose someone carries the stale €100 onto A_1 as though it were A_1 's price — treating the parent number as passing through unchanged — and then adds the fresh A_2 print:

$$\underbrace{1,000 \times 100}_{A_1, \text{ stale, assumed}} + \underbrace{1,000 \times 20}_{A_2, \text{ fresh}} = \text{€}120,000.$$

Against a true €100,000, that is a phantom €20,000 — and it is exactly A_2 's value counted twice, once embedded in the stale parent price and once as the fresh A_2 print. The ledger refuses it twice over. Identity is a declared operator, never assumed, so there is no conversion carrying A onto A_1 alone for the stale mark to ride; and single-basis consumption refuses mixing a stale pair value beside a fresh single-component print. Once both children have printed — A_1 at €80, A_2 at €20 — the basket bridge retires and the mark is component-wise: $1,000 \times 80 + 1,000 \times 20 = \text{€}100,000$, each on its own chain. The value crossed the boundary as a whole and came out a sum of parts only when the parts had prices of their own.

Cross-instrument transport is one atomic multi-unit transaction, and a mark is carried at the level the data actually supports — the pair, while only the pair is stamped — never fabricated per component from a price the notice could not have contained.

12 An Elective Merger

The hardest corporate action for a deterministic system is one where the holders choose. Different holders of the same security take different consideration, so the same event produces different moves for different wallets — and yet the security has one price series that every holder must read the same way. This stop, the last on the ACME line, shows both halves handled by holder choice entering as ordinary data. The following March, an acquirer Q takes over ACME; write the target line as T , held 1,000 shares, marked €100, split 500 to holder A and 500 to holder B .

12.1 Nothing moves until confirmation

The announcement opens a messy window: terms proposed, elections open, proration unknown. The design's answer to the window is to declare no operator for it. With no operator, nothing adjusts — no quote is transported, no strike moves, no number changes — and any attempt to consume a datum across the still-unresolved boundary fails closed. The fail-closed reaches only consumption that would *cross* the announced boundary; it does not freeze the T line. A fresh T print taken in the standing pre-merger basis and read in that same basis is no boundary crossing, so T marks normally all through the arbitrage window — only a datum trying to convert across the unresolved terms is stopped. The world has said a merger is coming; the log records that it is pending and computes nothing across it. This is the opposite of guessing, and it is what lets the window be messy without corrupting anything.

Confirmation resolves it. The final terms are €102 cash or 2.04 Q shares per T share, under a 60% aggregate cash cap that here does not bind. A elects cash, B elects stock, so half the book elects cash — comfortably under the cap, which is why nothing is prorated. The elections are not inferred; each holder's choice is a logged instruction, read by the contract as declared data.

12.2 Different moves, one fate, and a weld in aggregate

On the entitlement plane the two holders get different moves, because they chose differently: A 's 500 shares become $500 \times 102 = \text{€}51,000$ cash, and B 's 500 shares become $500 \times 2.04 = 1,020$ Q shares. On the quotation plane the T series has one fate, read identically by every holder: the confirmed blend of €51 cash and 1.02 Q per T share, declared as the composite $\text{Subst}(Q, 1.02) \circ \text{Shift}(-51)$ — no new case: the composite is one declared conversion, the pair $(f, c) = (1.02, 51)$. A stale €100 T mark transports across it to $(100 - 51)/1.02 = 2450/51$ per Q share, an exact rational.

No single holder's moves match the blend — that is what an election means — so the door cannot check the merger holder by holder against a uniform rule. It checks the legs in aggregate at the confirmed $(f, c) = (1.02, 51)$: the quantity legs must sum to f times the total, $1,020 = 1,000 \times 1.02$, and the cash legs to c times the total, $\text{€}51,000 = 1,000 \times 51$. Both hold. This aggregate weld is a distinct check reserved for a confirmed elective structure; it does not replace the strict per-holder weld that governs a uniform action, which stands at full strength for every non-elective boundary. Each holder's own allocation is then economic detail, checked by recomputation against that holder's logged election and the confirmed proration.

The value is continuous in aggregate to the minor unit. Before: $1,000 \times 100 = \text{€}100,000$. After: $1,020 \times 2450/51 + 51,000 = 49,000 + 51,000 = \text{€}100,000$, with no rounding residue, since $1,020/51 = 20$ exactly. The per-holder differences are each measured against the one transported mark of $2450/51$ per Q share, and each is exactly that holder's own elected consideration seen against that mark: A 's €51,000 of cash stands €1,000 above the €50,000 its 500 shares carried before, and B 's 1,020 Q shares, valued at the mark, come to €49,000, €1,000 below. The two differences sum to zero, but this is a frame identity, not a transfer between the holders: no value passes from B to A . Each holder's consideration settles against the acquirer Q — Q pays A 's cash and issues B 's shares — and because the cash cap does not bind, both elections are satisfied in full, nothing prorated and nothing exchanged between the two. The $\pm \text{€}1,000$ is simply what each election is worth when both are marked at the single blended price the T line resolves to.

And because the boundary transported only what it observed, a fresh Q price is ordinary marking profit and loss, not a phantom. If Q 's next print is €50, the transported mark $2450/51$ per share leaves a gap of $50 - 2450/51 = 100/51$ per share, and over 1,020 shares that is $1,020 \times 100/51 = \text{€}2,000$ exactly — old information meeting a fresh mark, shown as market movement, never absorbed into the merger operator. *Because holders elect differently, no single*

holder's moves match the confirmed blend, so the per-holder weld that governs a uniform action cannot apply; what the door checks instead is the aggregate weld — the quantity legs summing to f times the total, $1,020 = 1,000 \times 1.02$, and the cash legs to c times the total, $€51,000 = 1,000 \times 51$ — the distinct check an elective structure forces. That the contract stays deterministic and replays to the same per-holder transactions holds here too, but it is the aggregate weld this stop turns on.

13 Securities Lending in Brief

Every stop so far kept one thing fixed: a position is a signed number per unit, and ownership *is* that number. Securities lending is the one business that breaks the assumption, because it separates ownership from possession — a lender still owns shares that are physically in someone else's hands. This last stop shows the ledger absorbing that not by adding a new architecture, but by widening one coordinate.

This stop works the same firm's September special dividend — the €2 distribution the option stop already met — through a book that holds 1,000 ACME shares with 400 of them out on loan. A loan is opened before the dividend and recalled after. A scalar balance can no longer describe the book: it must show that it still owns 1,000 shares for the purpose of profit and loss, while only 600 are available to sell. So the balance generalises from a scalar to a vector of coordinates, per holder and unit — one coordinate for each distinct custody state a share can be in. There are five:

$$\vec{w} = (\text{own}, \text{on-loan}, \text{borrowed}, \text{collateral-posted}, \text{collateral-received}).$$

Each coordinate passes a physical-action test — a distinct action moves it and only it — and only *own* carries economic value and drives profit and loss. The list is open at the same seam it is closed: a deployment that permits re-pledging received collateral adds a *rehypothecated* coordinate the same way, one physical action moving one coordinate, and this stop does not, so five suffice. Available inventory is not a stored number at all; it is a projection, computed on read as $\text{avail} = \text{own} - \text{on-loan} + \text{borrowed}$, so it cannot drift out of step with the coordinates it is derived from. When ownership and possession coincide, on-loan and borrowed are zero, avail equals own, and the vector degenerates back to the scalar of every earlier stop.

Two other events sit at the edges of the loan. A *locate*, the confirmation a borrower obtains before selling short, enters as a pre-trade check: a stamped confirmation admitted before the sale, not a balance change. And each loan is its own unit, carrying its own fee, collateral, and reporting — so a loan is a first-class thing in the log, not an annotation on a share balance. The fee is not a figure kept on the side: it *accrues* over the loan's life, a daily fee on the on-loan notional booked as periodic cash moves on the loan unit, so the lender's fee income is itself a fold of recorded moves and reconciles by construction.

13.1 A loan, opened and recalled

Opening the loan is one atomic transaction, and the rule that keeps the vector consistent is the single-coordinate move principle: one atomic move writes exactly one coordinate of one unit, the *same* coordinate at both of its endpoints — it subtracts a quantity from that coordinate at the source wallet and adds the identical quantity to that coordinate at the destination ($\text{src}[c] -= q, \text{dst}[c] += q$). An operation that must change several coordinates is not one move but a *transaction*: a list of single-coordinate moves, admitted all-or-nothing. The open is such a transaction, of three moves.

Two carry the share leg, and each writes its own coordinate against a contra in the loan's per-relationship virtual wallet. The first marks the book's *on-loan* up by 400 — $\text{Book.onloan} += 400$

against Virtual.onloan $- = 400$ — recording that 400 of the book’s shares are now owned-but-lent. The second records the borrower’s possession — Borrower.borrowed $+ = 400$ against Virtual.borrowed $- = 400$. The virtual wallet thus carries a contra on *every* coordinate the transaction writes, which is exactly why the per-unit sum closes; a lone contra discharging only one of the two coordinates would leave the ledger 400 shares short of balancing. Through both moves the book’s *own* is never written and stays at 1,000, and the standing custodian leg that holds the book’s shares at $-1,000$ is likewise untouched. The third move is the collateral. Here the collateral is cash, and cash is value on the *own* coordinate: the move is Borrower.own $- = €21,000$, Book.own $+ = €21,000$ on the cash unit — 400 shares marked at the post-split €50 is €20,000 of exposure, and a 105% margin posts €21,000. It is a single move on one coordinate, like any cash transfer. The *collateral-posted*, *collateral-received*, and, where a deployment re-pledges, *rehypothecated* coordinates are named for the non-cash bilateral case, where the collateral is itself securities to be tracked in custody without being owned; cash collateral needs none of them, and the return obligation it creates is carried in the loan unit’s own state. After the open:

$$\vec{w}_{\text{book}}(\text{ACME}) = (1,000, 400, 0, \dots), \quad \vec{w}_{\text{borrower}}(\text{ACME}) = (0, 0, 400, \dots).$$

The book’s available inventory is $1,000 - 400 + 0 = 600$; the borrower’s is $0 - 0 + 400 = 400$. Crucially, *no move wrote the book’s own in ACME* — it stands at 1,000 through the whole loan, so the book has not derecognised the shares and its profit and loss still reads on 1,000. That the lender’s own is constant across lending is not a rule enforced after the fact; it holds by construction, because loan initiation writes on-loan and never own.

Conservation holds here in exactly the sense it held for a plain share, and it is worth separating the three statements it comprises, because a lent position needs all three.

Law A — per-unit balance. For each unit, the signed sum over *all* slots that carry it — the real wallets, the per-relationship virtual wallet, and the custodian contra — is zero; equivalently, every transaction nets to zero per unit. This holds only because the virtual wallet carries a contra on every coordinate a move writes. Summing the ACME slots the loan touches:

$$\underbrace{1,000}_{\text{book own}} \underbrace{-1,000}_{\text{custodian}} + \underbrace{400}_{\text{book on-loan}} \underbrace{-400}_{\text{virtual on-loan}} + \underbrace{400}_{\text{borrower borrowed}} \underbrace{-400}_{\text{virtual borrowed}} = 0.$$

The two virtual contras, one per share coordinate, are what close the sum; drop either and the line fails to balance by 400.

Law B — physical conservation. The shares physically possessed sum, across all parties, to the shares in issue — a constant N , here 1,000, not zero. Possession is own+on-loan+borrowed+collateral-received, so a lent share is counted once, at the borrower who holds it, through the lender’s minus: the book possesses $1,000 - 400 = 600$, the borrower 400, and $600 + 400 = 1,000$. (The collateral-received term is zero here, where the collateral is cash; it carries the non-cash case, where securities held as collateral are possessed without being owned.)

Law C — own is constant. The lender’s *own* is invariant across the loan by construction, because loan initiation writes *on-loan* and never *own*. This is the property just seen from the move list, stated as a law: it is why the book’s profit and loss keeps reading on 1,000 shares for the whole life of the loan.

13.2 The special dividend, split by payer

The loan is open across ACME’s September special dividend — the same €2 per share distribution that, in the option stop, moved a written strike. A plain holding takes one dividend; a lent holding takes two, and the coordinate is what tells them apart. The book owns 1,000 shares, but only 600 are in its possession; the other 400 sit on loan, held by the borrower on the record date.

The special dividend is a cash event — no share moves, only cash on the *own* coordinate — and it resolves into three cash moves. On the 600 it possesses, the book receives the *real* dividend, paid by ACME’s paying agent: $600 \times 2 = \text{€}1,200$, carrying a real dividend’s tax character — withholding and franking as the jurisdiction attaches them to a distribution from the company. On the 400 on loan, ACME pays that same real dividend to whoever holds the shares on the record date — the borrower, not the book: $400 \times 2 = \text{€}800$ from the paying agent to the borrower. The book is not left short, because the loan obliges the borrower to pass it a *manufactured* dividend of $400 \times 2 = \text{€}800$, borrower to lender.

This third leg is a different cash flow in every respect that matters. Its payer is the borrower, not the issuer; its tax character is a substitute payment rather than a distribution — typically carrying no dividend withholding credit, and in many jurisdictions taxed as ordinary income rather than a qualified dividend in the lender’s hands — and it is booked on the *on-loan* line, as its own unit and move, precisely because the on-loan coordinate is what records that these 400 shares are owned-but-lent and so owe a manufactured, not a real, dividend. Cash conserves across the three moves: the paying agent is out €2,000, the book is up €2,000, and the borrower nets zero — it received €800 from the issuer and passed €800 on.

The two legs sum to what ownership entitles the book to: €1,200 real plus €800 manufactured is €2,000, exactly $\text{€}2 \times 1,000$, because *own* stayed at 1,000 through the loan and *own* is what drives profit and loss. What the loan changed is not how much the book earns but who pays it and in what character — and that distinction rides on the coordinate, never smuggled into one dividend number. A scalar balance could not have told the real €1,200 from the manufactured €800; the vector does, because the on-loan coordinate was there to carry the difference.

13.3 Recall, with a deadline

The recall is not an instantaneous mirror image but a dated obligation, and it generates no moves at all: it is a notification that flips the loan from *active* to *recalled* and registers the obligation object the framework already carries. A recall sets a return deadline; the borrower returning the shares before that deadline is the discharge, and if the deadline passes with the shares not back, the lender’s *buy-in* — purchasing them in the market and charging the borrower the cost — is the compensation path. A deadline, a discharge predicate, a compensation action, and one of its terminal states reached by the deadline: this is the obligation object of the framework, applied to a loan rather than a coupon. The coordinate mirror fires on the return that settles the recall, not on the recall notice itself.

When the return does complete, it is the mirror of the open, move for move: Borrower.borrowed $\text{--} = 400$ against Virtual.borrowed $\text{+} = 400$; Book.onloan $\text{--} = 400$ against Virtual.onloan $\text{+} = 400$; and the €21,000 of cash collateral returns on the *own* coordinate, Book.own $\text{--} = \text{€}21,000$, Borrower.own $\text{+} = \text{€}21,000$. Afterwards the book’s ACME vector is $(1,000, 0, 0, \dots)$ and its available inventory is back to 1,000; the borrower’s line returns to the zero vector; the cash collateral is released in full. The available projection moved $1,000 \rightarrow 600 \rightarrow 1,000$ with no stored value that could fall out of step, because it was a fold of the moves the whole time.

Nothing else changed. The door is the same door, the checks are the same checks, and the lender’s and borrower’s differing views of the loan are two folds of the one log, not two records to reconcile. *Securities lending is accommodated by generalising the coordinate, not the architecture — the same door, the same conservation, and views as folds of the same log carry the position and entitlement mechanics of a loan: ownership held while possession moves, a dividend split by payer across the lent line, and a recall discharged or bought in against a deadline — and they collapse back to the scalar the moment ownership and possession coincide again.*

14 What the Full Framework Adds

The tour ran eight products through one machine and watched it hold. A book that runs this machine for real needs more than the machine, and it would be dishonest to close without saying what. This section names that additional machinery. The point worth making first is that none of it is a second architecture bolted to the side. Each piece is a consequence of the door, the log, and the views already built: a reporting obligation is a view, an industry mapping is a function over the log, a correctness claim is a check that runs against the log, the timer service is named infrastructure outside the trust boundary, and the standing scope limits are the same boundary the ledger reconciles against rather than governs. Where this account does not carry a piece to its full depth, it stops and says so, rather than implying the depth is not there.

14.1 Regulatory reporting is a projection, not a second record

A trading book must report its activity — trades, transactions, positions — to authorities that keep their own records. It is tempting to build this as a parallel pipeline: a second store of trade data, shaped for the report, kept in step with the ledger by a process that copies and reconciles. That design reintroduces exactly the failure the whole system exists to remove, because now there are two records of one activity and a way for them to disagree.

There is no need for it. The economic content of any such report — what happened, in what quantity, to what unit, when — is already in the move stream, because that stream is the record of what happened. A report is therefore one more view: a fold that reads the committed log and emits the report’s rows in the report’s shape. It computes; it never mutates state, never writes back, and holds no authority the log does not already hold. If two reports of the same activity must agree, they agree for the same reason the balance sheet and the profit-and-loss statement agree — they are two folds of one log, and there is only one log for them to fold. What a report needs beyond the economic content is reference data: the identifiers and classifications an authority requires — a trade’s assigned identifier, a party’s registered legal identifier, a counterparty’s regulatory class. Those are not facts the ledger forms; they are supplied at the boundary, by the authorities that own them, and joined to the projection as it is built. The ledger records the activity and projects it; it does not become the registrar of the identifiers the report is filed under.

14.2 Alignment to a common domain model

The industry describes a trade, and the events that happen to it over its life, in a shared domain model — a common vocabulary of products and lifecycle events that lets independent systems talk about the same trade and mean the same thing. The architecture is built to align with that vocabulary rather than to compete with it. The alignment is a structural correspondence, and it is concrete: the ledger’s unit corresponds to the model’s product — and, for a bespoke instrument, to the trade itself — and the map from the model’s business events to the ledger’s transactions is required to be total and deterministic. A trade becoming a position, a coupon falling due, a corporate action landing on a holding — each is an event in the model and a transaction at the door, and the map carries one to the other. Because the map is a function of the event alone, any conforming map inherits what the architecture already guarantees: it composes (a sequence of events maps to the sequence of transactions), it conserves (the moves it emits net to zero like any others), it keeps move order, and it is idempotent under the same keys the door already deduplicates. A corporate action is the one place the map does more than translate: the event supplies both halves of the transaction the ledger needs — the entitlement moves and the status write that advances the basis, with its declared conversion — so that the basis change, which a plainer mapping would discard as it had no field for it, is carried as

the first-class fact it is. This account states the alignment and its shape; it does not carry the field-by-field mapping, and does not pretend to.

14.3 The correctness claims are executable, not asserted

Throughout, this design has made claims of correctness: that quantity is conserved, that a price and a quantity meet in one basis and never two, that every product has exactly one declared reaction to every corporate-action class, that a producer's economics can be recomputed from the log and compared, that a dated obligation reaches a terminal state by its deadline. A claim like that is worth only as much as the reader's ability to check it, and the framework's discipline is that each such claim is not a sentence in prose but a property a machine re-checks against the reference behaviour. Every claim is a universally quantified statement — for every transaction, for every unit, for every boundary — and each is carried as an executable oracle: it is fed cases drawn from the closed enumerations of products and events the design admits, and it either holds on every case or names the case it fails on. Structural claims hold by construction, so their oracles confirm that the types admit no counterexample; economic claims are checked, so their oracles are the recomputation the witness performs. This is the concrete content, turned on the framework's own claims, of the discipline that a producer's output is trusted for nothing and verifiable in everything: a claim in this framework is something a machine re-checks, not something a reader is asked to take on trust, and coverage of the event vocabulary is a finite checklist rather than a hope that testing found the corner.

14.4 The execution engine and the liveness it underwrites

A lifecycle contract does not wake itself. Something must hold the dated obligations of every open position — an expiry, a coupon, an exercise window — and fire each when its time comes. That something is an execution engine, and it is infrastructure outside the trust boundary, named apart from the door, the log, and the views, holding no privileged access of any kind. It is bound by named requirements and nothing more: it must fire durably, so a timer survives a restart; it must deliver at least once, so a fire is never simply lost; and its firing is delivery, not data — it says *when*, while everything the event means is read from the log. Because the trigger is delivery and never data, an engine that stalls, fails over, fires early, or fires twice can delay a discharge but cannot corrupt the log: a late delivery submits the identical transaction, a duplicate delivery is absorbed once at the door, and an early or spurious fire, finding its observation or due-state not yet in the log, yields a refusal rather than a commit.

This is what lets the design carry liveness as a guarantee rather than a hope. Every contractual obligation is a first-class logged object, registered atomically with the position it binds and carrying three things: a deadline, a discharge predicate on ledger state, and a compensation action. The guarantee is that by its deadline every obligation reaches one of three terminal states — Discharged, Compensated, or Defaulted. The transition is itself an ordinary status write through the single writer for obligation state: the discharge predicate met on the discharging transaction sets Discharged; and once the deadline passes unmet, the engine wakes the obligation's compensation contract, which sets Compensated when the compensation action completes and Defaulted when even that cannot. No other component can move an obligation to a terminal state, because obligation state, like every other fact, has one legal writer. A coupon that should fire and does not is then not silence but a visible object past its deadline, which any fold of the log can see; a dated duty cannot be quietly forgotten, because forgetting it would be a state, and the state is detectable. The honest content of the guarantee is where it is conditional. It holds provided the engine is available at the deadline to deliver the trigger, the executor that runs the woken contract is available to act, any external cooperation the discharge requires is forthcoming, and the lifecycle function computes the right discharge from what it reads. Those conditions are named because they are real: the ledger cannot make a

counterparty pay, and cannot fire a timer while the engine that holds it is down. Under an engine outage the obligation is reached *eventually*, when delivery resumes — not instantaneously. What the log guarantees unconditionally is that it never loses the obligation; it only waits for the fire, and the wait is itself a recorded, detectable state.

14.5 What this framework does not do

Honesty about depth requires honesty about the edges. The framework has standing limits, and they are not gaps to be filled later but boundaries it is built to reconcile against rather than to govern. It does not form prices: what a structured note is worth is decided by a model on the far side of a wall the ledger never crosses, and the ledger records that number as a stamped observation without reproducing or validating it. It is not the settlement infrastructure: it projects committed activity outward as instructions, and the accounts that move real cash and securities are elsewhere. It is not a legal-agreement system: a contract in this design is the machine-readable form of a confirmation, not the confirmation's legal force. It is not a regulatory-submission channel: it computes the reporting view, and the filing of it is an external act. It is not a reference-data authority: the identifiers, classifications, and registrations it consumes are owned by others. Against each of these it reconciles at its boundary — the external record is independent, and reconciliation against it stays necessary, made detectable and narrow rather than eliminated.

Two residual trust assumptions remain, and naming them is part of the honesty. The first is at the ingest stamp: when market data enters, the stamp that says which basis the datum was disseminated in is trusted to be true of the source, and the architecture bounds what that trust buys — a datum crosses a boundary only by a declared operator, and a registered kind carries a mandatory invariance witness that pins exactly what the kind's declaration is trusted for, no wider. The second is the one the two-layer discipline has already been plain about: a contract's output is admitted before its economics are recomputed. A transaction that is well-formed but economically wrong — pays €98 where €100 was owed — is admitted, and its error is found afterward by the witness and repaired by a compensating transaction that names what it supersedes. Detection is therefore eventual, not preventive. What is guaranteed absolutely is that the structural invariants never break and that the wrong number is always findable from the log and detected whenever the witness runs; what is not claimed is that the wrong number is stopped before it is booked. How promptly the witness runs is a scheduling requirement the deployment owns, standing alongside the engine's. The design would rather book a detectable, repairable error than pretend to a prevention it cannot deliver.

15 Conclusion

15.1 The thesis, now shown

One claim carried this document: the ledger is the orchestrator of the output of many smart contracts. The contracts run outside its trust boundary. Each is triggered by an event, reads the declared data and stamped observations it is allowed to see, works out what must change, and produces transactions. The ledger runs none of that product-specific logic. It is four things and nothing more — a single door that admits or refuses each proposed transaction, an immutable log that records what the door admits, a family of views computed from the log, and one generic operator application at the read seam. Everything a product needs beyond those four lives in its contract, on the far side of the boundary.

That was asserted at the start. The tour discharged it. Eight economically distinct products crossed the same door: a cash equity through a split, a dividend forecast, a cleared future's daily margin, a structured note, a special dividend, a spin-off, an elective merger, and a securities

loan. No two share a payoff, a lifecycle, or a settlement convention. Yet none required product-specific logic inside the ledger; the two generalisations the tour did force — the aggregate weld and the custody coordinate — are product-agnostic widenings of an existing check and an existing coordinate. Each differed only in what its contract read and what it produced — the moves, terms versions, and status writes it emitted — and every one of those crossed the identical door, landed in the identical log, and was read back by the identical views. A structured note is not a special case of the machine; it is the same machine handed a different contract. Adding the ninth product is writing a ninth contract, not extending the ledger. The claim that the ledger holds nothing product-specific is now a demonstrated fact, because eight products passed through and the only generalisations they forced were product-agnostic.

Each stop also closed one design question, by showing the mechanism absorb a real product with real numbers rather than by argument. The eight closes are the load the thesis actually bears, and they are worth collecting in one place.

15.2 Each stop, and the choice it settled

Tour stop	Design choice it substantiated
Cash equity & split	Single-basis consumption composed in declared order: a quantity and a price meet in one frame only, fixed by the declared boundary and not by arrival order. An adjustment is a declared operator applied at read, never a restatement of stored shares.
Dividend forecast	A forecast enters as a stamped observation of a registered kind, one dimension per component — never as a model number the ledger must trust. The kind's invariance witness catches a misdeclared component before it can mark the position.
Listed futures	Variation margin is ordinary conserved transactions keyed per settlement day. There is no privileged variation-margin engine; the door that carries a share purchase carries a future's entire margin life, and initial margin sits outside as posted collateral.
Structured note	The model-agnostic wall: the pricing that decided the note's worth is never opened. Two layers protect the record — the door catches transactions that are broken, recomputation from the log catches the ones that are merely wrong.
Special dividend	The adjustment schedule's three layers — class default, declared override, designated discretion — behind a totality gate that fails closed. Bspokeness is a higher-precedence layer of declared data, not a branch in code.
Spin-off	Cross-instrument transport is one atomic multi-unit transaction, and a mark is carried only at the level the data supports — the pair while only the pair is stamped — never fabricated per component from a price the notice could not contain.
Elective merger	Holder choice enters as logged data. The per-holder weld cannot apply when holders elect differently, so the door checks the aggregate weld instead; the contract stays deterministic and replays to the same per-holder transactions.
Securities lending	Generalise the coordinate, not the architecture. The same door, the same conservation, and views as folds of the same log carry a loan's position and entitlement mechanics, collapsing back to a scalar when ownership and possession coincide.

Read down the right column and one pattern is unmistakable. Every choice pushes product-specific knowledge out of the ledger and into declared data a contract reads: the split's frame, the forecast's dimensions, the special dividend's override, the merger holder's election. The ledger's own conduct never varies across the eight. It admits, records, and reads back — the same three acts, on every product.

15.3 What the whole design rests on

The reason eight products could share one machine is that a small set of invariants holds by construction rather than by care, and every product inherits them for free. An unconserved transaction cannot be built, because a move is a transfer and has no way to express a quantity appearing from nowhere; conservation is therefore not checked but guaranteed. A transaction applies in full or not at all, so no product can leave the log in a half-written state. The log

is append-only, so a recorded fact is never overwritten — an adjustment is a new entry read through a declared operator, never a restatement of an old one. There is one door, so there is one place where well-formedness is decided and one place a proposal can be refused. Defaults fail closed: an unregistered datum kind is unconsumable, a stale datum met with no declared conversion is a typed refusal, an incomplete adjustment schedule blocks registration. And the model-agnostic wall keeps every pricing decision on the contract’s side of the boundary, entering the ledger only as a stamped observation the door never interprets. These are the properties that made the tour uneventful. Each is enforced once, at the door or in the types, and no contract — careful, careless, or hostile — can reach around them.

What remains is to state these commitments in the compact, formal register they deserve, drawing the guarantees together as one synthesis.

15.4 The formal synthesis

Nothing here is new; each statement compresses a guarantee the preceding sections established in plain terms.

Conservation is inherited from the generator. A move subtracts a quantity from one wallet and adds the same quantity to another: its net effect on a unit’s total across all wallets is zero, and a transaction, a finite list of moves, nets a sum of zeros. Formally, net change per unit is a homomorphism from move sequences under concatenation to quantities under addition, fixed by its values on the generators.

Proposition 15.1. *Every move nets to zero, so the net-change map is the zero homomorphism — it factors through the trivial group — and no expressible transaction can create or destroy quantity.*

The log is free, and every view is a fold out of it. State holds no information the event sequence does not: replay to any point rebuilds the exact state that held then, and every view — balance, valuation, profit and loss, report — is a fold of the one log, determined by what it does with a single transaction. Two folds of one log cannot disagree about a fact recorded once; the reconciliation guarantee of Section 1 is this uniqueness.

Proposition 15.2. *A view is the unique homomorphism from sequences of admitted transactions under concatenation to state transformations under composition, extending its per-transaction action; replay is therefore deterministic, independent of where it is cut, and re-reads history rather than re-executing any producer.*

Conversion is an arrow that exists or does not. A datum crosses a basis boundary only by the operator the boundary declared, from a closed menu; omission declares nothing, and the crossing is refused.

Proposition 15.3. *For each datum kind, between two bases on one chain there is at most one conversion — the composite of the declared operators between them, in effective order — and a datum crosses exactly when that composite exists; fail-closed is the absence of the arrow.*

A contract is a recomputable function. A contract is a deterministic, total function from the committed log prefix and its trigger to one result — a proposed transaction or a typed refusal. Structural soundness, once admitted, is Proposition 15.1 plus the door’s checks; economic soundness is checkable because everything the function read is in the log: two evaluations of one function on one input are equal.

Proposition 15.4. *Economic correctness is the agreement of the recorded transaction with an independent re-evaluation of the producing function on the same committed prefix, compared in canonical form: a discrepancy locates a producer defect exactly, and the bound is eventual detection and recorded repair, never admission-time verification.*

One arrow, three names. A trade, a lifecycle event, and a corporate action differ in what fires them and in what they read — never in shape. Each is one arrow from log prefix and trigger to a proposed transaction or a typed refusal; every guarantee above is stated over that arrow’s inputs and output, never over which instance fired, and so holds for every instance, including those yet to be written.

One record, freely generated by what the door admits; conservation the zero homomorphism on its generators; every view a fold out of it; every conversion the one arrow its chain declares; every producer a recomputable function into it. Not five disciplines maintained in parallel: one structure, read five ways.